

DOCUMENTATION DE CONCEPTION DU MODELE IA SONGO SRN

Architecture proposee, representation relationnelle du Songo, algorithmes du reseau, boucle MCTS - self-play, strategie de corpus, protocole experimental et feuille de route.

Position scientifique du document.

Le SRN est l'architecture proposee, et non une conclusion deja demontree.

La documentation distingue explicitement :

- les decisions d'architecture deja retenues ;
- les hypotheses experimentales ;
- les parametres a calibrer ;
- les tests et ablations susceptibles de confirmer ou de remettre en cause le choix relationnel.

Principe directeur :

Le moteur sait comment le Songo fonctionne ; l'intelligence artificielle apprend comment bien y jouer.

Version : v1.4 - revision de coherence architecture / RL / dataset / code

Date : 23 septembre 2026

Fichier : songo/docs/Documentation_Modele_IA_Songo_SRN_v1.4_corrige.pdf

Table des matieres

- I. Presentation du projet
- II. Nature structurelle du Songo
- III. Concepts strategiques: ZT, TS, TR, TM, Bidoua
- IV. Architecture SRN proposee
- V. Architecture mathematique
- VI. Graphe Songo et relations
- VII. Policy-Value Network
- VIII. Interface moteur - reseau - recherche
- IX. MCTS et PUCT
- X. Strategie d'entrainement
- XI. Dataset d'apprentissage et contrat d'entrainement
- XII. Reward, nulles, exploration et curriculum
- XIII. Architecture du code existant et adaptation cible
- XIV. Squelette de code
- XV. Protocole experimental
- XVI. Validation de l'intelligence strategique
- XVII. Decisions, hypotheses, validations
- XVIII. Feuille de route
- Annexe A. Hyperparametres initiaux
- Annexe B. Differences avec le code actuel
- Annexe C. Algorithmes recapitulatifs

I. Presentation generale du projet

I.1. Contexte et origine du projet

Le Songo est un jeu de strategie dans lequel la decision ne repose pas uniquement sur l'etat instantane du plateau. La distribution des graines dans les cases determine des possibilites de circulation, de transfert, de retour, de capture et d'accumulation dont les consequences peuvent apparaitre plusieurs coups plus tard. Une position doit donc etre appreciee non seulement a partir du nombre de graines presentes dans chaque case, mais aussi a partir des transformations que ces graines peuvent provoquer au cours du jeu.

Cette caracteristique constitue le point de depart du present projet. Le developpement de la version numerique du Songo a d'abord conduit a la realisation d'un moteur de jeu deterministe, charge d'appliquer les regles: validation des coups, semis, circulation des graines, captures, transmission, gestion de la famine, changement de joueur et determination de la fin d'une partie. Ce moteur constitue la representation operationnelle du jeu. A partir d'un etat S et d'une action legale a , il permet de calculer exactement l'etat resultant S' .

Une premiere intelligence artificielle a ensuite ete construite selon une approche classique de recherche dans l'arbre de jeu, notamment a travers les algorithmes Minimax/Negamax avec elagage Alpha-Beta. Cette approche explore plusieurs suites de coups possibles, evalue les positions obtenues et selectionne une action. Elle a progressivement ete enrichie par des connaissances specifiques au Songo et par des optimisations destinees a ameliorer la profondeur et la qualite de la recherche.

Cette premiere generation a permis d'obtenir un adversaire artificiel fonctionnel et de constituer une base importante d'experimentation. Elle presente cependant une limite fondamentale pour l'objectif poursuivi maintenant: l'intelligence strategique depend encore largement de ce que le programmeur a explicitement introduit dans l'algorithme. La fonction d'evaluation, les criteres de comparaison des positions, les heuristiques et certaines connaissances strategiques traduisent necessairement une conception humaine de ce qui constitue une bonne ou une mauvaise position. Le systeme recherche efficacement a l'interieur de ce cadre, mais il ne construit pas encore lui-meme sa propre representation strategique du Songo.

Le projet entre donc dans une seconde etape. Il ne s'agit plus seulement d'augmenter la profondeur du Minimax ou d'ajouter de nouvelles heuristiques, mais de rechercher les conditions permettant a une intelligence artificielle d'apprendre par l'experience du jeu. L'objectif est qu'elle puisse jouer un grand nombre de parties, observer les consequences de ses decisions, confronter differentes strategies et ameliorer progressivement son comportement sans que chaque principe strategique doive etre prealablement traduit en regle informatique.

Cette orientation est particulierement importante au regard de la nature strategique du Songo. Les travaux conceptuels mobilises dans le projet mettent en evidence une lecture dynamique du plateau a travers la circulation des graines et des notions telles que ZT, TS, TR et TM. Ils introduisent egalement les notions de Bidoua, Yinda-Bidoua et Transport-Bidoua, qui montrent que la valeur strategique d'une case ne peut pas etre deduite de son seul nombre de graines. Ces concepts doivent cependant servir ici a comprendre le probleme et a evaluer l'intelligence obtenue, et non necessairement a fournir directement au futur modele les reponses qu'il devrait apprendre.

Cette distinction conduit a un principe central du projet: separer les regles du jeu de l'intelligence qui joue au jeu. Le moteur conserve la responsabilite de tout ce qui peut etre determine exactement: legalite d'un coup, circulation des graines, captures, transmission, famine, etats terminaux et calcul des etats successeurs. Le modele d'apprentissage n'a donc pas vocation a redécouvrir les regles du Songo. Son role est different: il doit apprendre a evaluer les situations produites par ces regles et a developper progressivement une politique de decision strategique.

Le passage a l'apprentissage automatique ne signifie pas pour autant que l'architecture du futur modele soit deja connue. Le choix d'un reseau relationnel, d'un reseau convolutif, d'un MLP ou d'une autre representation ne doit pas etre effectuee uniquement parce qu'une architecture est couramment utilisee dans les jeux. La conception doit partir de la structure propre du Songo: qu'est-ce qu'une case? quelles informations portent les graines qu'elle contient? quelles relations existent entre les cases? comment représenter l'amont, l'aval, les circulations longues, les retours et les transformations produites par un coup? C'est a partir de ces questions que devra etre determinee l'architecture capable d'apprendre efficacement.

Le projet s'oriente ainsi vers une approche d'apprentissage par renforcement, dans laquelle le modele pourra progressivement produire ses propres experiences par le jeu, notamment par auto-jeu. Les parties generees

constitueront alors la matiere premiere de son apprentissage. Le modele ne sera plus seulement entraine a reproduire les decisions d'un moteur expert: il devra pouvoir ameliorer sa strategie a partir des resultats de ses propres confrontations.

Dans cette perspective, les 10 000 positions de Songo deja constituees occupent une place precise. Elles fournissent un laboratoire de positions reelles pour examiner des situations de jeu, tester des representations, verifier le comportement des algorithmes et construire des cas d'evaluation. Elles ne constituent pas le corpus d'entrainement du futur modele autonome. Le futur corpus d'apprentissage devra etre produit par le processus d'apprentissage lui-meme: self-play, recherche, replay buffer, adversaires historiques et positions critiques controlees.

Le present travail nait donc du passage entre deux generations d'intelligence artificielle pour le Songo: une premiere generation construite autour d'un moteur de recherche Minimax/Negamax et de connaissances strategiques introduites par le concepteur; puis une seconde generation dans laquelle les regles restent garanties par le moteur, mais ou la representation strategique et la politique de decision doivent etre progressivement apprises par l'experience.

Idee directrice. L'enjeu n'est pas simplement de remplacer Minimax par un reseau neuronal. Il est de determiner quelle representation informatique du Songo permettra a un systeme d'apprentissage de decouvrir, exploiter et eventuellement depasser les strategies actuellement codees a la main, tout en restant strictement conforme aux regles du jeu.

1.2. Problematique

Le developpement d'une intelligence artificielle capable de jouer efficacement au Songo pose une difficulte qui depasse la simple recherche du meilleur coup a partir d'une position donnee. Le moteur actuel permet deja de connaitre exactement les coups legaux, d'executer le semis, de determiner les captures, de gerer la transmission et la famine et, grace a Minimax/Negamax, d'explorer les consequences de plusieurs coups successifs. Le probleme n'est donc plus principalement de savoir comment faire jouer une machine au Songo, mais de determiner comment lui permettre d'en apprendre progressivement la strategie.

Dans l'approche actuelle, la qualite de la decision depend en partie de la fonction d'evaluation concue par le programmeur. Des elements tels que les graines acquises dans les magasins, la mobilite ou certaines approximations de territoire strategique permettent au moteur de comparer les positions. Cette approche est efficace pour guider la recherche, mais elle introduit necessairement une representation humaine de ce qu'est une position favorable. Le moteur Minimax/Negamax explore et evalue avec une heuristique; le modele autonome recherche doit, lui, apprendre progressivement une representation et une preference strategiques.

Or, le Songo presente une difficulte particuliere: la valeur strategique d'une position n'est pas reductible a la valeur individuelle de ses cases. Une case contenant un certain nombre de graines peut produire des consequences tres differentes selon sa position, les cases situees en amont, celles qu'elle pourra atteindre, la possibilite d'un retour, la configuration du territoire adverse ou encore la reponse que l'adversaire pourra apporter. La succession, l'amont, l'aval, la frontiere et la portee longue entre les cases ne sont donc pas des details de representation: ce sont des elements qui participent directement a la valeur strategique d'une position.

Il en resulte qu'une representation elementaire du plateau sous la forme d'un simple vecteur de nombres peut contenir toute l'information brute necessaire sans pour autant rendre explicite la structure qui donne un sens strategique a cette information. Deux configurations numeriquement proches peuvent conduire a des dynamiques de jeu tres differentes en fonction des trajectoires de semis, des transferts, des retours et des transformations qu'elles rendent possibles.

Cette difficulte apparait egalement dans les concepts strategiques associes au Songo. Les phenomenes de transfert, de retour, de transferts multiples, de Bidoua, de Yinda-Bidoua ou de Transport-Bidoua suggerent qu'une accumulation de graines ne possede pas une valeur fixe et independante de son environnement. La modification d'une seule case peut transformer les possibilites de circulation et modifier la valeur strategique de plusieurs autres cases. Le futur modele doit donc pouvoir apprendre des dependances spatiales, relationnelles et temporelles, sans que toutes ces relations strategiques lui soient donnees sous forme d'heuristiques predefiniees.

Une deuxieme difficulte apparait alors. Si l'on fournit directement au reseau les concepts strategiques deja elabores, par exemple une mesure calculee du Bidoua ou une evaluation provenant du Minimax, le modele risque de devenir essentiellement un apprenant des connaissances du systeme existant. Il pourrait reproduire efficacement les decisions du moteur expert sans necessairement decouvrir une representation strategique propre. A l'inverse, si

aucune structure pertinente du jeu ne lui est fournie, le reseau devra consacrer une partie importante de son apprentissage a reconstruire des relations que le moteur connait deja exactement.

Le probleme consiste donc a determiner ou placer la frontiere entre ce qui doit etre calcule et ce qui doit etre appris. Les regles deterministes du jeu n'ont pas besoin d'etre redécouvertes. Pour un etat S et un coup legal a , le moteur sait calculer exactement:

$$S' = T(S, a)$$

ou T represente la fonction de transition du Songo. Le reseau n'a donc aucune raison d'apprendre approximativement le semis, la capture ou la legalite d'un coup lorsque ces informations peuvent etre obtenues sans erreur par le moteur. En revanche, le moteur ne doit pas decider a la place du modele quelle strategie est la meilleure. C'est precisement cette connaissance que l'on souhaite faire emerger de l'apprentissage.

Une troisieme difficulte concerne la nature de l'apprentissage lui-meme. Un apprentissage exclusivement supervise a partir des decisions du Minimax conduirait naturellement le modele a chercher a reproduire le comportement de ce dernier. Or, l'ambition du projet est differente: le Minimax peut servir de reference, d'adversaire ou d'instrument d'evaluation, mais ne doit pas constituer la verite strategique definitive que le nouveau modele serait charge d'imiter. Le systeme doit progressivement pouvoir produire ses propres experiences de jeu, apprendre des resultats de ses parties et confronter ses nouvelles strategies a celles de ses versions precedentes et du moteur existant.

La problematique comporte donc simultanement trois dimensions:

- **representationnelle:** comment represente une position de Songo de maniere suffisamment riche pour rendre accessibles les relations entre les cases et la circulation des graines;
- **decisionnelle:** comment apprendre a evaluer les positions et a selectionner les actions sans encoder directement dans le reseau les conclusions strategiques du programmeur;
- **d'apprentissage:** comment permettre au modele d'ameliorer cette connaissance par l'experience et l'auto-jeu, tout en utilisant le moteur deterministe pour garantir l'exactitude des regles.

Formulation centrale. Comment concevoir une intelligence artificielle capable d'apprendre de maniere autonome la dynamique et les strategies propres au Songo, a partir des regles exactes du jeu et de l'experience acquise par le jeu, sans dependre d'une fonction d'evaluation strategique entierement definie par le programmeur?

Cette problematique generale entraine une question d'architecture particulierement importante: quelle representation de l'etat du Songo et quelles relations entre les cases faut-il mettre a la disposition du modele pour lui permettre d'apprendre les effets de circulation, de transfert et de temporalite du jeu, tout en laissant emerger de l'apprentissage les notions proprement strategiques?

Elle entraine egalement une question d'autonomie: comment organiser l'apprentissage par renforcement et le self-play afin que le modele puisse progressivement ameliorer sa politique de jeu, etre evalue independamment du systeme Minimax existant et, eventuellement, decouvrir des strategies que celui-ci ne contient pas?

Ces trois questions - represente, apprendre et decider - structurent la suite de la documentation. Elles ont un avantage important: elles ne presupposent pas encore que le SRN, le GNN, le MCTS ou une architecture particuliere constituent la bonne reponse. Les sections suivantes devront construire puis tester cette reponse.

I.3. Objectif general

L'objectif general du projet est de concevoir, developper et valider une intelligence artificielle capable d'apprendre progressivement et de maniere autonome a jouer au Songo, en s'appuyant sur un moteur garantissant l'application exacte des regles du jeu et sur un mecanisme d'apprentissage permettant au modele de construire sa propre representation strategique a partir de l'experience.

Il ne s'agit donc pas uniquement de developper une intelligence artificielle capable d'obtenir de bonnes performances face a un joueur humain ou au moteur Minimax/Negamax existant. L'ambition est de mettre en place un systeme dans lequel la competence strategique n'est plus principalement determinee par une fonction d'evaluation concue manuellement, mais emerge progressivement des parties jouees, des consequences des decisions prises et de l'amelioration successive du modele.

Le moteur du Songo conserve, dans cette approche, la responsabilite des connaissances deterministes: legalite des actions, semis, circulation des graines, captures, transmission, famine, calcul des etats successeurs et conditions de

fin de partie. Le futur modele d'intelligence artificielle doit, quant a lui, apprendre a interpreter strategiquement les etats produits par ce moteur, a evaluer leurs perspectives et a privilegier les actions susceptibles de conduire a de meilleurs resultats.

L'objectif implique egalement de rechercher une representation de l'etat adaptee aux caracteristiques propres du Songo. Le modele devra etre capable d'exploiter non seulement la quantite de graines presente dans chaque case, mais aussi les relations entre les cases et les effets potentiels de la circulation des graines, sans que les conclusions strategiques telles que Bidoua, Yinda-Bidoua ou Transport-Bidoua lui soient necessairement imposees comme des regles de decision.

A terme, le systeme recherche devra pouvoir generer une part essentielle de ses propres experiences d'apprentissage par auto-jeu, ameliorer successivement sa politique de decision et etre confronte a differentes references, notamment ses versions anterieures et le moteur Minimax/Negamax, afin de mesurer objectivement la progression de ses capacites. Dans cette perspective, le Minimax peut demeurer un instrument de comparaison ou d'evaluation, mais il ne constitue pas la verite strategique finale.

Formulation synthetique. Concevoir et valider une intelligence artificielle autonome du Songo capable d'apprendre, par l'experience et l'auto-jeu, une politique de decision et une representation strategique adaptees a la dynamique propre du jeu, tout en s'appuyant sur un moteur deterministe garantissant l'application exacte de ses regles.

Cette formulation reste volontairement generale. Elle ne presuppose ni SRN, ni GNN, ni MCTS. Ces elements seront etudies comme des solutions candidates pour atteindre l'objectif, et non comme l'objectif lui-meme.

I.4. Objectifs specifiques

Pour atteindre l'objectif general, le projet poursuit les objectifs specifiques suivants.

OS1 - Formaliser le Songo comme environnement d'apprentissage

Definir une representation informatique rigoureuse de l'etat du jeu, des actions possibles, des transitions et des etats terminaux, tout en maintenant le moteur de regles comme autorite deterministe pour la legalite des coups, le semis, les captures, la transmission, la famine et le calcul des resultats. Le reseau ne doit ni modifier l'etat, ni determiner la legalite d'un coup, ni calculer les transitions.

OS2 - Identifier les caracteristiques structurelles du Songo pertinentes pour l'apprentissage

Etudier l'organisation du plateau, la circulation des graines, les relations entre cases, les phenomenes de transfert et de retour ainsi que les dependances a courte et longue portee, afin de determiner quelles informations doivent etre representees explicitement et lesquelles peuvent etre decouvertes par le modele au cours de son apprentissage. L'enjeu est notamment de depasser une lecture strictement locale ou purement numerique du plateau.

OS3 - Concevoir et comparer plusieurs representations et architectures neuronales

Developper des architectures candidates capables de traiter l'etat du Songo et comparer experimentalement leurs performances. Une representation vectorielle simple pourra servir de reference, tandis que des representations tenant davantage compte des relations entre les cases seront etudiees. L'objectif n'est donc pas de considerer a priori le SRN comme la solution definitive, mais de verifier experimentalement si une architecture relationnelle apporte effectivement un avantage.

OS4 - Permettre au modele d'apprendre a evaluer les positions et a selectionner les actions

Developper un mecanisme d'apprentissage permettant a l'intelligence artificielle d'estimer la qualite d'une situation de jeu et d'attribuer des preferences aux differents coups legaux, sans lui demander de reapprendre les mecanismes deterministes deja maitrises par le moteur. Le modele devra ainsi apprendre la strategie, et non les regles du jeu.

OS5 - Mettre en place un processus d'apprentissage par renforcement fonde principalement sur l'auto-jeu

Permettre au modele de produire progressivement ses propres experiences en jouant contre lui-meme, d'observer les resultats obtenus et d'utiliser ces experiences pour ameliorer ses decisions au fil des generations. Le corpus principal d'apprentissage devra ainsi provenir des parties produites par le systeme lui-meme et non des 10 000 positions actuellement disponibles, qui restent avant tout un ensemble de positions de laboratoire et d'evaluation.

OS6 - Integrer un mecanisme de planification permettant d'anticiper les consequences des decisions

Etudier l'interet d'associer au modele appris un mecanisme de recherche permettant d'examiner plusieurs continuations possibles avant de jouer. Cette dimension est particulierement importante au Songo, ou la valeur d'un coup peut dependre d'une circulation ou d'une reponse adverse apparaissant plusieurs coups plus tard. Le MCTS/PUCT constitue actuellement une hypothese forte, mais son apport devra etre mesure experimentalement plutot que considere comme acquis.

OS7 - Favoriser l'emergence d'une connaissance strategique propre au modele

Eviter autant que possible d'imposer au reseau une fonction d'evaluation reproduisant directement les connaissances du moteur expert. Les notions de ZT, TS, TR, TM, Bidoua, Yinda-Bidoua et Transport-Bidoua pourront servir a analyser les comportements appris, a construire des situations de test ou, si necessaire, a experimenter des taches auxiliaires, sans etre considerees d'emblee comme des reponses que le modele doit recevoir en entree.

OS8 - Evaluer objectivement la progression et la robustesse de l'intelligence artificielle

Mettre en place un protocole experimental permettant de mesurer les performances du modele contre differentes references: architectures de base, versions anterieures du modele, moteur Minimax/Negamax et ensembles de positions critiques. L'evaluation ne devra pas reposer uniquement sur le taux de victoire, mais egalement examiner la qualite de l'evaluation des positions, la stabilite des decisions et le comportement du modele dans des situations strategiques caracteristiques.

OS9 - Verifier que les performances obtenues resultent reellement des choix d'architecture et d'apprentissage

Realiser des experiences controlees et des etudes d'ablation afin de determiner l'utilite reelle des differentes composantes introduites: relations entre cases, mecanismes de circulation, profondeur du reseau, recherche, eventuelles taches auxiliaires et strategie d'entrainement. Il s'agit ainsi de pouvoir expliquer pourquoi une architecture fonctionne mieux qu'une autre et non simplement constater qu'elle gagne davantage.

OS10 - Preparer une architecture exploitable dans le jeu Songo existant

Concevoir le systeme de maniere modulaire afin de separer clairement le moteur de regles, le modele neuronal, la recherche, l'auto-jeu, l'entrainement et l'evaluation. A terme, le modele valide devra pouvoir etre integre a l'application Songo, notamment dans l'environnement Unity, sans remettre en cause le moteur de regles existant.

Synthese. Ces objectifs specifiques forment une chaine logique: comprendre le Songo, le formaliser, determiner comment le représenter, concevoir plusieurs architectures, apprendre par l'experience, planifier, s'améliorer par self-play, comparer, valider, puis intégrer. A ce stade, la documentation ne declare toujours pas que le SRN est la solution: elle definit d'abord ce que la future intelligence artificielle doit etre capable d'accomplir.

I.5. Positionnement du projet: IA, apprentissage par renforcement et genie logiciel

Le projet se situe a l'intersection de trois domaines complementaires: l'intelligence artificielle, l'apprentissage par renforcement et le genie logiciel. Ce positionnement est important, car la realisation d'un joueur autonome de Songo ne releve pas uniquement de l'entrainement d'un reseau neuronal. Elle suppose aussi de formaliser le jeu, de concevoir une architecture d'apprentissage adaptee a sa structure et de construire un systeme logiciel capable de faire fonctionner ensemble le moteur de regles, le modele, la recherche, l'auto-jeu et les mecanismes d'evaluation.

1. Positionnement en intelligence artificielle

Dans sa première version, l'intelligence du jeu repose principalement sur une approche d'IA de recherche, dans laquelle Minimax/Negamax explore les suites de coups possibles et utilise une fonction d'évaluation pour comparer les positions. Le système actuel constitue donc déjà une intelligence artificielle capable de prendre des décisions dans le Songo.

La nouvelle étape du projet poursuit toutefois un objectif différent. Il ne s'agit plus seulement de rechercher efficacement dans un arbre de jeu dont l'évaluation a été définie par le concepteur. Il s'agit de construire une intelligence capable d'apprendre progressivement une représentation des situations et une politique de décision.

Le passage recherche peut ainsi être résumé de la manière suivante:

```
IA fondée sur recherche + heuristiques
->
IA fondée sur apprentissage + expérience + planification
```

Cette évolution ne signifie pas l'abandon du moteur existant. Au contraire, elle conduit à mieux séparer les responsabilités. Le moteur demeure responsable des règles exactes du Songo, tandis que le modèle apprend la valeur stratégique des situations et les préférences entre les actions possibles. Cette séparation entre moteur, réseau et recherche constitue l'un des principes fondamentaux de la conception.

L'enjeu scientifique porte alors sur la représentation de la connaissance stratégique. Le modèle devra progressivement apprendre qu'une position ne se résume pas au nombre de graines contenues dans les cases. Il devra exploiter les dépendances entre celles-ci, les possibilités de circulation et les conséquences différées des décisions. Le projet relève ainsi de l'apprentissage de représentations appliquées à un jeu de stratégie: transformer un état brut du Songo en informations utiles à la décision.

2. Positionnement en apprentissage par renforcement

Le projet relève plus précisément de l'apprentissage par renforcement, puisque le modèle doit apprendre à prendre des décisions successives dans un environnement et améliorer son comportement en fonction des résultats obtenus.

Dans cette formulation, le Songo constitue l'environnement. À un instant donné, l'agent observe un état S_t , choisit une action légale a_t , puis le moteur applique exactement cette action et produit un nouvel état:

$$S_{t+1} = T(S_t, a_t)$$

Le processus se poursuit jusqu'à atteindre un état terminal. Le résultat final de la partie fournit alors le signal d'apprentissage principal:

```
z = +1  victoire
      0  partie nulle
      -1 défaite
```

Cette orientation est importante: la récompense principale provient du résultat réel de la partie, et non d'une note stratégique intermédiaire entièrement construite à partir des heuristiques du moteur Minimax. Le but est d'éviter de transformer l'évaluation du système expert en vérité que le nouveau modèle serait simplement chargé d'imiter.

L'apprentissage devra notamment permettre au modèle d'acquiescer deux capacités complémentaires:

```
pi_theta(a | S)  politique: préférence pour les actions disponibles
V_theta(S)       valeur: estimation des perspectives associées à l'état
```

L'auto-jeu, ou self-play, occupe alors une place centrale. Le modèle pourra jouer contre lui-même, produire des trajectoires nouvelles, observer leurs résultats et utiliser ces expériences pour améliorer ses paramètres. Une nouvelle version pourra ensuite produire de nouvelles parties, donnant naissance à un processus itératif:

```
Jouer -> Produire des expériences -> Apprendre -> Evaluer -> Jouer à nouveau
```

Cette boucle correspond à une succession de checkpoints, de parties de self-play, d'un replay buffer, d'un entraînement et d'une évaluation avant promotion ou rejet du nouveau modèle. L'objectif est donc bien celui d'une IA qui améliore progressivement son comportement à partir de l'expérience qu'elle produit elle-même.

3. Positionnement en génie logiciel

La dimension de génie logiciel est tout aussi importante. Le projet ne consiste pas uniquement à expérimenter un algorithme d'apprentissage dans un environnement isolé. Il vise la réalisation d'un système d'intelligence artificielle complet, modulaire, testable, reproductible et susceptible d'être intégré au jeu Songo existant.

Plusieurs composants logiciels doivent fonctionner ensemble:

Le premier enjeu de genie logiciel consiste a assurer une separation claire des responsabilites. Le moteur de regles doit rester independant du reseau neuronal. Il doit pouvoir etre teste separement afin de garantir que deux etats identiques soumis a une meme action produisent exactement la meme transition. Le reseau ne doit jamais pouvoir modifier les regles, declarer lui-meme qu'un coup est legal ou simuler approximativement un semis.

Le deuxieme enjeu concerne la modularite de l'architecture. Les composants consacres a l'environnement, au reseau, a la recherche, au self-play, a l'entrainement et a l'evaluation doivent pouvoir evoluer independamment. Cette organisation permettra notamment de remplacer une architecture neuronale par une autre sans reecire le moteur du Songo, ou de comparer deux methodes de recherche en conservant le meme modele.

Cette modularite possede egalement une importance experimentale: pour determiner si une amelioration provient reellement du modele, de la representation des etats ou de la recherche, ces composants doivent pouvoir etre actives, remplaces ou supprimes independamment.

Le troisieme enjeu est celui de la reproductibilite. Chaque experimentation devra etre associee a une configuration identifiable: version du modele, hyperparametres, budget de recherche, generation du self-play, checkpoint utilise et protocole d'evaluation. La performance d'une version de l'IA devra ainsi pouvoir etre reproduite et comparee objectivement a celle d'une autre.

Enfin, le systeme devra etre concu en tenant compte de sa destination finale. L'intelligence artificielle n'a pas vocation a rester uniquement dans un environnement experimental Python. Une fois valide, le modele devra pouvoir etre integre au jeu Songo developpe sous Unity, avec un contrat d'inference stable et des contraintes compatibles avec une utilisation reelle.

4. Articulation des trois dimensions

Ces trois dimensions ne constituent donc pas trois projets independants. Elles correspondent a trois niveaux d'un meme systeme.

L'intelligence artificielle repond a la question: quelle representation et quel mecanisme de decision peuvent permettre a la machine de comprendre strategiquement une position de Songo?

L'apprentissage par renforcement repond a la question: comment cette intelligence peut-elle acquerir et ameliorer cette competence par l'experience plutot que par la seule programmation d'heuristiques?

Le genie logiciel repond a la question: comment construire un systeme fiable, modulaire et reproductible permettant a cette intelligence d'apprendre, d'etre evaluee puis d'etre integree au jeu reel?

Le projet Songo peut ainsi etre positionne comme un projet de conception d'un systeme intelligent autonome pour un jeu de strategie, dans lequel le genie logiciel fournit l'infrastructure fiable, l'intelligence artificielle fournit les mecanismes de representation et de decision, et l'apprentissage par renforcement fournit le processus par lequel cette intelligence acquiert progressivement sa competence.

Ce positionnement est essentiel pour la suite de la documentation: le choix eventuel du SRN, d'un GNN ou du MCTS ne definit pas la nature du projet. Ce sont des solutions techniques candidates a l'interieur d'un probleme plus general de conception d'une intelligence artificielle autonome adaptee aux caracteristiques particulieres du Songo.

1.6. Perimetre et demarche de conception

La presente documentation ne considere pas encore l'architecture SRN comme une verite demontree. Elle la traite comme une hypothese de conception issue de l'analyse du Songo, a comparer a d'autres representations. La demarche retenue consiste donc a partir du jeu, formaliser les contraintes, proposer une architecture, puis valider ou corriger cette proposition par experiences, ablations et evaluation sur positions critiques.

II. Nature structurelle du Songo

1. Etat du jeu

Une position operationnelle de Songo est representee par 14 cases jouables, deux magasins et le joueur au trait. Dans le moteur de reference, les indices physiques restent :

- cases 0..6 : joueur 1 ;
- cases 7..13 : joueur 2 ;
- magasin 14 : joueur 1 ;
- magasin 15 : joueur 2.

Etat brut :

$S = (x_0, \dots, x_{13}, \text{store_P1}, \text{store_P2}, \text{turn})$

La canonicalisation du point de vue du joueur au trait est une REPRESENTATION CANDIDATE pour le reseau, et non une propriete a supposer du moteur. Avant de l'utiliser, il faut verifier experimentalement que toutes les regles du moteur legacy sont symetriques sous cette transformation. Tant que cette verification n'est pas achevee, l'etat brut et le joueur au trait doivent etre conserves sans ambiguite dans le dataset.

Representation canonique candidate :

X_{self} = sept cases du joueur au trait
 X_{opp} = sept cases adverses
 $\text{stores} = (\text{store_self}, \text{store_opp})$
 turn = implicite uniquement apres validation de la symetrie

2. Circulation des graines

Le Songo est d'abord un jeu de circulation. Jouer une case ne consiste pas seulement a vider une case et a capturer localement. Le coup produit un trajet, traverse des zones, peut revenir, modifie les nombres de graines en cascade, cree ou detruit des possibilites de capture et change parfois la capacite de transmission.

Cela rend insuffisante une lecture uniquement locale du plateau. Un vecteur brut contient l'information necessaire, mais ne rend pas explicites les dependances de circulation que le modele devra apprendre.

3. Relations spatiales et dynamiques

Les relations candidates comprennent :

- la succession dans le sens de semis ;
- l'amont immediat ;
- les dependances de circulation calculees par le moteur ;
- les effets de portee longue produits par le nombre de graines.

Les relations NEXT et PREV constituent la geometrie minimale retenue pour la baseline relationnelle.

Les relations MIRROR, BORDER et DYNAMIC_SOWING restent des hypotheses a valider par ablation.

Point important.

La question n'est pas : "utiliser un GNN parce que le Songo est relationnel".

La question est : "quelles relations exactes du Songo doivent etre exposees au modele, et lesquelles doivent etre laissees a l'apprentissage ?"

4. Consequence pour l'IA

Le reseau doit pouvoir relier la quantite de graines d'une case a sa position, aux cases susceptibles de l'alimenter, aux cases qu'elle peut atteindre et aux transformations futures. Le graphe est une hypothese de representation de ces dependances ; son utilite devra etre demontree face aux baselines vectorielles.

III. Concepts strategiques : ZT, TS, TR, TM, Bidoua

1. ZT, TS, TR et TM : etats dynamiques de circulation

Les categories ZT, TS, TR et TM ne doivent pas etre definies comme de simples "zones" fixes du plateau. Elles decrivent des regimes dynamiques lies au trajet potentiel des graines d'une case.

ZT - Zero Transfert

Etat dans lequel le jeu de la case ne produit pas de transfert vers le territoire adverse.
Le mouvement reste dans le regime sans transfert.

TS - Transfert Simple

Le semis produit un transfert vers le territoire adverse et s'y termine sans retour vers l'amont du territoire d'origine au cours de ce mouvement.

TR - Transfert Retour

Le trajet franchit le territoire adverse puis revient vers le territoire d'origine. La valeur de la case depend donc d'une circulation qui ne s'arrete pas au premier transfert.

TM - Transferts Multiples

Le nombre de graines permet plus d'un tour complet du circuit. Lors d'un tour complet, la case d'origine n'est pas resemencee ; la circulation poursuit ensuite ses transferts. Ce regime correspond aux fortes accumulations et participe a la logique des greniers.

Ces definitions servent d'abord de vocabulaire d'analyse et de validation. Elles ne sont pas imposees comme features strategiques d'entree dans la version initiale du SRN.

2. Bidoua

Le Bidoua ne doit pas etre reduit a "une case sure" ni au seul seuil de cinq graines. Dans le moteur expert actuel, le bonus `safe_territory` (cases ≥ 5) est une approximation operationnelle utile, mais il ne constitue pas une definition complete du Bidoua.

Dans la documentation de conception, le Bidoua est traite comme un concept strategique lie a la liberte temporelle, aux coups favorables disponibles et a la capacite de "mastiquer le temps". Deux accumulations comparables peuvent donc avoir des valeurs differentes selon leur environnement et les reponses disponibles.

3. Yinda-Bidoua et Transport-Bidoua

Yinda-Bidoua :

diminution du potentiel de Bidoua provoquee par l'ajout de graines a une case. L'effet est non lineaire : une seule graine supplementaire peut modifier fortement la circulation future.

Transport-Bidoua :

modification ou reduction du potentiel de Bidoua produite par le transport de graines depuis des cases en amont. La valeur d'une accumulation est donc relationnelle et depend de la maniere dont elle est alimentee.

4. Usage dans le projet

ZT/TS/TR/TM, Bidoua, Yinda-Bidoua et Transport-Bidoua servent prioritairement :

- a construire des positions critiques ;
- a analyser les representations apprises ;
- a verifier si le modele distingue des regimes de circulation differents ;
- a definir, si necessaire, des taches auxiliaires experimentales.

Ils ne sont pas des labels principaux du dataset RL et ne sont pas imposes comme entrees strategiques en v0.1.

IV. Architecture SRN proposee

1. Vue generale

La question centrale n'est pas seulement de savoir quel type de reseau neuronal utiliser, mais de definir precisement ce que le reseau doit voir, transformer et produire. Pour le Songo, le reseau ne doit pas apprendre a jouer les graines: le moteur connait deja exactement le semis, les captures, les coups legaux et les transitions. Le reseau doit apprendre ce que signifie strategiquement l'organisation des graines dans une position.

Une position canonique peut etre notee:

```
S = (x_0, x_1, ..., x_13, M_self, M_opp)
```

Le moteur sait calculer exactement $T(S, a) = S'$. Le reseau doit donc apprendre deux fonctions complementaires:

```
P_theta(a | S)  quels coups meritent d'etre consideres
V_theta(S)      qualite strategique de la position pour le joueur au trait
```

Le Songo Relational Network est l'architecture proposee pour apprendre ces deux fonctions a partir de 14 representations de cases, de relations propres au plateau et d'un contexte global minimal.



2. Transformation generale du reseau

L'algorithme neuronal propose se lit comme une suite de transformations:

```
S
-> encodage des cases
-> relations Songo
-> propagation relationnelle
-> representation globale
-> Policy(S) et Value(S)
```

Une subtilite importante concerne la Policy. Chaque action correspond a jouer l'une des sept cases du joueur au trait. Il est donc preferable que le score de l'action depende directement de la representation finale de cette case, combinee au contexte global du plateau:

```
score(a) = f_policy(h_a, h_S)
```

Cette formulation est plus adaptee au Songo qu'une tete qui produirait simplement sept logits a partir d'un resume global:

```
(score_0, ..., score_6) = MLP(h_S)
```

La tete Policy devient ainsi action-conditionnee: elle evalue chaque coup a partir de la case effectivement jouee et de la situation globale.

3. Construction des 14 noeuds

Chaque case jouable devient un noeud du graphe:

```
C_0, C_1, ..., C_13
```

Le noeud ne contient pas seulement le nombre brut de graines. Il recoit un vecteur descriptif minimal:

```
X_i = [n_i, camp_i, position_i, distance_frontiere_i]
```

n_i represente le nombre de graines normalise. Les autres composantes indiquent la position de la case dans le plateau canonique et son appartenance relative. Dans la premiere version, le reseau ne recoit pas directement

Bidoua, Yinda-Bidoua, Transport-Bidoua, une valeur Minimax, un score heuristique ou le meilleur coup d'un teacher. Ces notions doivent servir d'abord a l'analyse et a l'evaluation, afin d'eviter de donner au modele une interpretation strategique deja fabriquee.

4. Encodage individuel des cases

Chaque vecteur X_i est transforme par un petit encodeur neuronal partage entre les cases:

$$h_i^0 = \text{NodeEncoder}(X_i)$$

Conceptuellement:

$$X_i \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow h_i^0$$

Si la dimension latente retenue est 64, chaque case devient un vecteur de $\mathbb{R}^{64}$. On obtient donc une representation 14×64 . A ce stade, chaque case est comprise individuellement: elle sait quelle position elle occupe, a quel camp elle appartient et combien de graines elle contient, mais elle ne connait pas encore ses relations avec les autres cases.

5. Relations structurelles et dynamiques du Songo

Le coeur de l'architecture est la construction du graphe. Une premiere famille de relations est structurelle et certaine:

$$\begin{aligned} \text{NEXT: } C_i &\rightarrow C_{i+1} \\ \text{PREV: } C_{i+1} &\rightarrow C_i \end{aligned}$$

Ces relations decrivent le sens de circulation et la possibilite de remonter vers l'amont. Elles constituent la geometrie minimale du plateau.

La seconde famille est dynamique. Pour chaque case jouable i , le moteur peut determiner exactement quelles cases seraient affectees si cette case etait jouee. On peut alors introduire une relation:

$$\begin{aligned} C_i &\text{ --dynamic_sowing--} \rightarrow C_j \\ \text{si les graines de } C_i &\text{ peuvent atteindre } C_j \text{ lorsque } i \text{ est jouee} \end{aligned}$$

Cette relation est beaucoup plus riche qu'un simple voisinage: une meme case contenant une graine ou quinze graines ne produit pas le meme graphe dynamique. Elle constitue donc une hypothese experimentale majeure. Elle semble naturelle pour le Songo, mais elle devra etre testee par ablation plutot qu'acceptee comme une verite acquise.

6. Message passing Songo

Les blocs relationnels permettent a chaque case de recevoir des informations des cases auxquelles elle est reliee. Pour chaque type de relation r , un message est calcule:

$$m_{\{j \rightarrow i\}^r} = \phi_r(h_j^k, h_i^k)$$

Chaque relation possede ses propres parametres. Le reseau peut donc apprendre qu'une information venue d'une case amont, d'une case voisine ou d'une case atteinte par un semis potentiel n'a pas le meme sens strategique.

$$\begin{aligned} m_i &= \sum_r \sum_{\{j \in N_r(i)\}} m_{\{j \rightarrow i\}^r} \\ h_i^{k+1} &= \text{LayerNorm}(\\ &\quad h_i^k + \text{Update}(h_i^k, m_i, c) \\ &\quad) \end{aligned}$$

Plusieurs couches de propagation sont necessaires pour représenter des dependances plus longues. Une couche capture surtout les relations directes; deux couches permettent de représenter des dependances de voisins de voisins; trois ou quatre couches peuvent commencer a modeliser des phenomenes d'amont, de preparation et de transport. Les valeurs $K = 2, 3, 4$ devront donc etre comparees experimentalement.

7. Contexte global et representation du plateau

Certaines informations appartiennent au plateau entier plutot qu'a une case. La premiere version doit rester prudente et inclure uniquement les informations fondamentales:

$$g = [M_{\text{self}}, M_{\text{opp}}, \text{seeds_in_play}]$$

Ces valeurs sont normalisees puis encodees:

$$c = \text{GlobalEncoder}(g)$$

Après K propagations, on dispose de $h_0^K, \dots, h_{13}^K$. Il faut ensuite construire une représentation globale h_S . Une moyenne simple est une baseline utile, mais elle peut écraser les cases stratégiquement exceptionnelles. Il faudra donc comparer:

```
MeanPooling
AttentionPooling
```

L'attention permettrait au réseau d'apprendre que certaines cases deviennent plus importantes que d'autres selon la position. La représentation globale finale est:

```
h_S = concat(h_pool, c)
```

8. Têtes Policy et Value

La tête Value reçoit h_S et produit une estimation bornée:

```
V_theta(S) = tanh(ValueMLP(h_S))
```

La tête Policy, elle, est calculée action par action. Pour chaque case jouable a dans $0..6$:

```
q_a = PolicyMLP(concat(h_a^K, h_S))
```

Le moteur fournit ensuite le masque légal. Les coups interdits reçoivent un score très négatif, puis la distribution est normalisée:

```
q_a = -infty si a est illegal
P_theta(a | S) = exp(q_a) / sum_{b in A(S)} exp(q_b)
```

Le réseau produit donc:

```
Network(S) = (P_theta(S), V_theta(S))
```

La Policy indique les coups qui méritent d'être explorés; la Value estime la qualité de la position. Le choix final peut ensuite être affiné par un mécanisme de planification comme MCTS.

9. Algorithme conceptuel complet

```
Algorithme SONGO_NETWORK(S)
Entree:
  S = position courante du Songo
Sorties:
  P = probabilités des actions
  V = valeur estimée de la position

1. Le moteur canonicalise S.
2. Le moteur calcule LegalMask(S).
3. Pour chaque case i = 0..13:
  X_i = EncodeRawCase(S, i)
  h_i = NodeEncoder(X_i)
4. Construire les relations structurelles:
  NEXT
  PREV
5. Construire éventuellement les relations dynamiques
  de circulation à partir du moteur.
6. Pour k = 1..K:
  Pour chaque relation r:
    calculer les messages entre cases.
  Pour chaque case i:
    agréger les messages recus.
    h_i = Update(h_i, messages)
7. Encoder les informations globales:
  c = GlobalEncoder(stores, seeds_in_play)
8. Construire la représentation du plateau:
  h_pool = Pool(h_0, ..., h_13)
  h_state = concat(h_pool, c)
9. VALUE:
  V = tanh(ValueMLP(h_state))
10. POLICY:
  Pour chaque case jouable a = 0..6:
    score[a] = PolicyMLP(concat(h_a, h_state))
11. Pour chaque action illégale:
    score[a] = -infty
12. P = softmax(score)
13. Retourner P, V
```

10. Pourquoi pas un MLP seul?

Un MLP sur 16 compteurs peut apprendre des corrélations statistiques et servir de baseline. Il est même nécessaire pour vérifier que le SRN apporte un gain réel. Mais il n'exprime pas explicitement la circulation, l'amont, l'aval ou les distances de semis. Il laisse au réseau la charge de reconstruire toute la géométrie du jeu à partir de positions entières.

11. Pourquoi pas figer AlphaZero comme réponse finale?

La boucle Policy-Value + MCTS + self-play est une inspiration forte, mais elle ne suffit pas à définir une architecture scientifique. Le document propose SRN + MCTS comme hypothèse principale, puis prévoit des expériences: MLP, SRN sans MCTS, SRN avec relations réduites, budgets MCTS différents, tâches auxiliaires, curriculum et évaluation sur positions critiques.

12. Architecture action-conditionnée et valeur des successeurs

En plus de $V(S)$, le système peut évaluer un coup par son successeur exact:

$$Q_{\theta}(S, a) \approx V_{\theta}(T(S, a))$$

$T(S, a)$ est calculé par le moteur de règles, pas par le réseau.

Cette convention est adaptée au Songo parce que les conséquences exactes d'un coup sont riches, mais parfaitement calculables. Le réseau n'a pas besoin d'apprendre le semis; il doit apprendre la valeur des positions qui en résultent.

13. Lecture algorithmique de la décision

```
Algorithme SRN_Decision(S)
Entree:
  S: position Songo brute
Sortie:
  action choisie

1. S_canon, legal_mask = moteur.canonicalize(S)
2. G = GraphBuilder.build(S_canon)
3. logits, value = SRN.forward(G, legal_mask)
4. priors = softmax(logits masques)
5. si mode jeu rapide:
   action = argmax(priors)
   sinon:
     pi_mcts = MCTS.search(S_canon, priors, value)
     action = sample_ou_argmax(pi_mcts)
6. retourner action
```

Cette séparation rend le système testable: si une décision est mauvaise, on peut isoler le problème dans la représentation, dans les poids du réseau, dans le budget MCTS ou dans le protocole d'entraînement.

V. Architecture mathématique

1. Représentation de l'état

Soit un plateau canonique composé de 14 cases jouables. Pour chaque case i , on construit un vecteur:

```
x_i = [
  graines_i / 70,
  propriétaire_i,
  position_i,
  distance_frontiere_i,
  indicateurs_optionnels_i
]
```

Le contexte global contient au minimum:

```
g = [
  store_self / 70,
  store_opp / 70,
  seeds_in_play / 70
]
```

V. Architecture mathematique

1. Representation de l'etat

Pour chaque case i , on construit un vecteur minimal :

$x_i = [\text{graines}_i / 70, \text{proprietaire_relatif}_i, \text{position}_i, \text{distance_frontiere}_i]$

Le contexte global minimal est :

$g = [\text{store_self} / 70, \text{store_opp} / 70, \text{seeds_in_play} / 70]$

La mobilite, une phase de jeu, Bidoua, Yinda, une menace ou une evaluation Minimax ne sont pas introduits automatiquement. Ils restent des variantes experimentales.

2. Graphe relationnel

Le graphe est :

$G(S) = (V, \{E_r\}, X, g)$

$V = \{0, \dots, 13\}$

En $v0.1$, les relations DECIDEES sont :

$R_{\text{base}} = \{\text{next}, \text{prev}\}$

Relations CANDIDATES a tester :

$R_{\text{exp}} = \{\text{mirror}, \text{border}, \text{dynamic_sowing}\}$

Les notions upstream, downstream et long_reach ne sont pas des types d'aretes imposes en $v0.1$. Elles peuvent emerger de plusieurs propagations NEXT/PREV ou etre testees ulterieurement comme variantes.

3. Message passing

Pour chaque relation r :

$m_i^k = \text{somme}_{\{j \text{ dans } N_r(i)\}} \phi_r(h_j^k, h_i^k, e_{\{j,i\},r})$

Puis :

$h_i^{k+1} = \text{LayerNorm}(h_i^k + \psi(h_i^k, m_i^k, c))$

Le nombre de blocs K est un hyperparametre. Les valeurs 2, 3 et 4 seront comparees.

4. Politique, valeur et Q

$h_{\text{pool}} = \text{Pool}(\{h_i^K\})$

$h_{\text{state}} = \text{concat}(h_{\text{pool}}, c)$

Pour chaque action locale a dans $0..6$:

$\text{logits}_a = \text{PolicyHead}(\text{concat}(h_a^K, h_{\text{state}}))$

$p_{\text{theta}}(a|S) = \text{softmax}(\text{mask_legal}(\text{logits}))_a$

$V_{\text{theta}}(S) = \text{tanh}(\text{ValueHead}(h_{\text{state}}))$

$Q_{\text{theta}}(S,a)$ reste optionnel :

$Q_{\text{theta}}(S,a) \sim V_{\text{theta}}(T(S,a))$

ou peut etre implemente par une tete action-conditionnee dediee.

5. Loss

Baseline scalaire :

$$L = \lambda_p * \text{CE}(\pi_{\text{MCTS}}, p_{\text{theta}}) \\ + \lambda_v * \text{MSE}(z, V_{\text{theta}}) \\ + \lambda_{\text{reg}} * R(\text{theta})$$

Variante experimentale :

remplacer la Value scalaire par une tete WDL et une cross-entropy WDL.

Dans le self-play, π_{MCTS} est la distribution de visites et z est le resultat terminal reel.

V. Architecture mathematique - suite

6. Algorithme de forward du SRN

Algorithme SRN_Forward(G, legal_mask)

Entrees :

G.X : features des 14 cases
G.g : contexte global minimal
G.E_r : aretes par relation active
legal_mask : actions legales locales

Sorties :

policy_logits / p_theta
V_theta

1. pour chaque case i :
 $h_i^0 = \text{NodeEncoder}(x_i)$
2. c = GlobalEncoder(g)
3. pour k = 0 .. K-1 :
 pour chaque relation active r :
 pour chaque arete j -> i dans E_r :
 $m_{\{j,i,r\}} = \text{Message}_r(h_j^k, h_i^k)$
 pour chaque noeud i :
 $m_i = \text{somme des messages entrants}$
 $h_i^{k+1} = \text{LayerNorm}(h_i^k + \text{Update}(h_i^k, m_i, c))$
4. h_pool = Pool($h_0^K, \dots, h_{13}^K$)
5. h_state = concat(h_pool, c)
6. V_theta = tanh(ValueHead(h_state))
7. pour chaque action locale a dans 0..6 :
 logits[a] = SharedPolicyMLP(concat(h_a^K , h_state))
8. logits[a illegal] = -inf
9. p_theta = softmax(logits)
10. retourner p_theta, V_theta

7. Signification des tenseurs

X	: [14, node_dim]	representation initiale des cases
g	: [global_dim]	magasins + graines encore en jeu
E_r	: [2, n_edges_r]	aretes source/destination d'une relation active
H^K	: [14, hidden_dim]	representations finales des cases
h_state	: vecteur global	pooling des noeuds + contexte global
policy_logits	: [7]	scores avant softmax
p_theta	: [7]	probabilites sur coups legaux
V_theta	: [1]	valeur scalaire du point de vue de la representation

Remarque.

La ligne "g = magasins, phase, mobilite" de la version precedente est supprimee : phase et mobilite ne font pas partie du contexte minimal retenu. Elles restent des variantes a tester.

VI. Graphe Songo et relations

1. Relations minimales v0.1

Relation	Description	Statut
next	Case suivante dans le sens de semis canonique.	Decide v0.1
prev	Case precedente, utile pour l'amont immediat.	Decide v0.1
mirror	Case correspondante dans le camp adverse.	A valider
border	Relation proche de la frontiere entre territoires.	A valider
dynamic_sowing	Cases atteintes par le semis de la case source dans l'etat courant.	Hypothese forte
capture_zone	Cases adverses a 2-4 graines susceptibles d'etre capturees.	Optionnelle

2. Relations statiques et dynamiques

Les relations statiques sont constantes pour toutes les positions. Les relations dynamiques dependent du nombre de graines. Les deux familles ne servent pas le meme objectif: les premieres donnent la geometrie du plateau, les secondes donnent le potentiel de circulation actuel.

Relations statiques:
i -> i+1, i -> i-1, i -> mirror(i)

Relations dynamiques:
i -> j si j est touchee par le semis actuel depuis i
i -> j si j est une case adverse capturable apres arrivee
i -> j si j devient une case de retour ou de transport

3. Pourquoi ne pas injecter directement Bidoua/Yinda?

Il y a deux strategies possibles. La premiere ajoute Bidoua, Yinda et Transport-Bidoua comme features calculees. Elle peut accelerer l'apprentissage, mais elle risque de transformer le reseau en imitateur de notre analyse. La seconde expose les relations de circulation et observe si les concepts emergent. La proposition SRN privilegie la seconde voie, avec des taches auxiliaires optionnelles pour les comparer.

VI. Graphe Songo et relations - suite

4. Algorithme de construction du graphe

Algorithme BuildGraph(S, relation_config)

Entree :

S : etat brut ou canonique valide
relation_config : relations experimentales activees

Sortie :

G : graphe relationnel

1. creer 14 noeuds, un par case jouable
2. pour chaque case i :
 x_i = encode_case_minimal(S, i)
3. encoder le contexte global minimal :
 g = encode_global(store_self, store_opp, seeds_in_play)
4. ajouter les relations de base :
 next : i -> successeur_structuel(i)
 prev : i -> predecesseur_structuel(i)
5. si relation_config.mirror :
 ajouter les aretes mirror
6. si relation_config.border :
 ajouter les relations de frontiere definies par le protocole experimental
7. si relation_config.dynamic_sowing :
 pour chaque action legale i :
 trajet = moteur.trace_sowing(S, i)
 pour chaque case jouable j effectivement touchee :
 ajouter i -> j dans dynamic_sowing
8. retourner G = (X, g, E_next, E_prev, E_exp...)

La relation dynamic_sowing est calculee a partir du moteur exact. Elle n'est pas une heuristique strategique : elle expose un fait de transition. Son utilite pour l'apprentissage reste cependant une hypothese a mesurer.

5. Niveau de confiance des relations

NEXT et PREV :

decisions v0.1 ; geometrie minimale.

MIRROR et BORDER :

hypotheses ; a conserver uniquement si les ablations montrent un gain.

DYNAMIC_SOWING :

hypothese forte ; elle injecte une information de trajet exacte mais peut etre redondante avec ce que plusieurs couches relationnelles apprennent deja.

VII. Policy-Value Network

1. Encodeur de noeuds

Chaque case recoit un vecteur normalise minimal. Les magasins restent dans le contexte global afin de distinguer les graines encore circulantes des graines deja acquises.

2. Message passing

Les blocs relationnels propagent l'information entre cases. Le reseau doit apprendre a exploiter les relations fournies sans recevoir directement les conclusions strategiques Bidoua/Yinda.

3. Policy Head

La Policy est ACTION-CONDITIONNEE. Chaque action correspond a une case du joueur au trait.

Le meme petit reseau SharedPolicyMLP est applique a chacune des sept actions :

score_a = SharedPolicyMLP(concat(h_a^K, h_state))

Cette formulation remplace toute implementation qui produirait directement sept logits a partir du seul h_state.

6. Algorithme de Policy Head

```
Algorithme PolicyHead(H_nodes, h_state, legal_mask)
Entrees:
  H_nodes = representations finales  $h_0^K \dots h_{13}^K$ 
  h_state = representation globale du plateau
  legal_mask = coups legaux fournis par le moteur

1. pour a dans 0..6:
    logits[a] = PolicyMLP(concat(H_nodes[a], h_state))
2. pour a dans 0..6:
    si legal_mask[a] = faux:
        logits[a] = -inf
3. pi = softmax(logits)
4. retourner pi
```

Le masque legal est essentiel: il empeche le reseau de gaspiller de la capacite a apprendre la legalite, qui est deja connue exactement. La Policy apprend seulement la preference entre coups possibles.

7. Algorithme de Value Head

```
Algorithme ValueHead(h_state)
1. y = MLP(h_state)
2. v = tanh(y)
3. retourner v

Convention:
  v proche de +1: position favorable au joueur courant
  v proche de 0: position nulle ou incertaine
  v proche de -1: position defavorable au joueur courant
```

Pendant le backup MCTS, la valeur change de signe a chaque changement de joueur. Cette convention evite d'avoir deux Value Heads separees ou une valeur absolue dependante du joueur physique.

VIII. Interface moteur - reseau - recherche

Information	Producteur	Consommateur	Regle
Etat canonique	Moteur	SRN	Entree stable du reseau.
Coups legaux	Moteur	Policy/MCTS	Jamais appris par le reseau.
Successeur $T(S,a)$	Moteur	MCTS et Q action-conditionne	Transition exacte.
Prior $P(a S)$	SRN	MCTS	Guide l'exploration.
Valeur $V(S)$	SRN	MCTS/entrainement	Estime l'issue.
Resultat terminal z	Moteur	Training	Cible Value principale.

Pourquoi conserver le moteur exact?

Une erreur de transition dans le Songo peut inverser une sequence. Un semis mal calcule change la case d'arrivee, la capture, la famine ou la transmission. Le moteur de regles doit donc rester independant du reseau, deterministe et teste.

Contrat d'inference

Contrat Environment -> Network

Input:

state_canonical: 16 compteurs ou structure equivalente

legal_mask: 7 booleans

graph: representation relationnelle derivee de state_canonical

Output:

policy_logits: 7 reels avant softmax

value: reel dans [-1, +1]

Interdits:

le reseau ne modifie jamais l'etat

le reseau ne declare jamais un coup legal ou illegal

le reseau ne calcule jamais une transition

IX. MCTS et PUCT

1. Role de MCTS

Le reseau seul repond a "que vaut cette position?". MCTS repond a "que se passe-t-il si ce coup est joue puis defendu?". Au Songo, cette difference est importante parce que beaucoup de coups ne revelent leur vraie valeur qu'apres une circulation ou une reponse adverse.

2. Formule PUCT

$$\text{Score}(S, a) = Q(S, a) + c_{\text{puct}} * P(S, a) * \sqrt{N(S)} / (1 + N(S, a))$$

$Q(S,a)$ est la valeur moyenne observee par les simulations. $P(S,a)$ vient du SRN. $N(S,a)$ est le nombre de visites du coup. Le moteur calcule tous les successeurs.

3. Cibles produites par MCTS

A chaque position de self-play, la distribution des visites devient la cible de Policy. Cela fournit un signal plus riche qu'un simple meilleur coup one-hot.

$$\pi_{\text{MCTS}}(a | S) = N(S,a)^{(1/\tau)} / \sum_b N(S,b)^{(1/\tau)}$$

4. Bruit et exploration

Pendant l'apprentissage, un bruit de Dirichlet peut etre ajoute a la racine pour eviter que le reseau ne repete trop vite ses propres preferences.

$$P_{\text{root}} = (1 - \epsilon) * P_{\text{network}} + \epsilon * \text{Dirichlet}(\alpha)$$

5. Algorithme MCTS detaille

```
Algorithme MCTS_Search(S_root, network, simulations)
Entree:
  S_root: position courante
Sortie:
  pi_mcts: distribution de visites

1. initialiser racine avec S_root
2. evaluer racine:
   P, V = network(S_root)
   P = ajouter_bruit_dirichlet(P) si self-play
   expand(root, P)
3. repeter simulations fois:
   node = root
   path = [root]
   tant que node est developpe et non terminal:
     choisir a maximisant Q(node,a) + U(node,a)
     node = child(node,a)
     ajouter node a path
   si node est terminal:
     value = resultat_terminal(node.state)
   sinon:
     P, value = network(node.state)
     expand(node, P)
     backup(path, value)
4. pour chaque action a:
   pi_mcts[a] = visits(root,a) / somme_visites
5. retourner pi_mcts
```

6. Backup et changement de perspective

```
Algorithme Backup(path, value)
1. pour node dans path inverse:
   node.N += 1
   node.W += value
   node.Q = node.W / node.N
   value = -value # changement de joueur
```

Le changement de signe est necessaire parce que $V(S)$ est toujours exprimee du point de vue du joueur au trait dans S . Quand on remonte d'un enfant vers son parent, le point de vue change.

X. Strategie d'entrainement

1. Principe retenu: self-play, RL et recherche

Il faut distinguer deux methodes d'entrainement qui ne poursuivent pas exactement le meme objectif. Entrainer le reseau sur des couples (S, best_action_Minimax) releve principalement de l'apprentissage supervise: le modele apprend a imiter les decisions produites par un teacher. Cette approche peut etre utile, mais elle ne doit pas devenir la methode centrale du projet, car l'objectif final est de construire une intelligence capable d'ameliorer sa strategie par sa propre experience.

La methode principale retenue pour l'IA autonome du Songo est donc:

```
Self-play + reinforcement learning + recherche
```

Le reseau joue des parties, produit des experiences, ces experiences alimentent un replay buffer, puis l'entrainement produit un nouveau reseau. Ce nouveau reseau genere ensuite de nouvelles parties. Le dataset d'apprentissage n'est donc pas un corpus fixe: c'est un corpus vivant, produit par le systeme au fil des generations.

```
Network_0 -> D_1 -> Network_1 -> D_2 -> Network_2 -> D_3 -> ...
```

2. Pourquoi le dataset Minimax ne doit pas devenir le corpus principal

Le dataset existant contient des champs utiles comme best_action, policy_target, wdl_target et des evaluations associees au teacher/Minimax. Si l'on entraine principalement le nouveau reseau avec ces donnees, l'objectif implicite devient:

```
S -> imiter Minimax
```

Ce n'est pas inutile: cela peut produire une baseline, verifier le pipeline ou fournir un warm-start. Mais ce n'est pas le coeur de l'architecture cible. Les cibles de politique du dataset sont souvent proches d'un one-hot, et les cibles de valeur proviennent du teacher plutot que du resultat reel d'une partie jouee par l'agent. Elles ne doivent donc pas etre confondues avec les labels produits par le self-play.

3. Format du vrai dataset RL

Au depart, le reseau Network_theta0 peut etre initialise aleatoirement. Sa politique est peu informative et sa valeur n'est pas encore calibree. Le moteur Songo garantit cependant les regles: il interdit les coups illegaux, applique les transitions exactes et detecte la fin de partie.

Pendant une partie de self-play, a chaque position S_t, une recherche comme MCTS utilise le reseau et le moteur pour produire une distribution de visites pi_t:

```
pi_t = [0.03, 0.08, 0, 0.61, 0.17, 0.11, 0]
```

Cette distribution devient la cible de politique. A la fin de la partie, le resultat terminal donne la cible de valeur:

```
z_t = +1 si le joueur au trait dans S_t gagne  
z_t = 0 si la partie est nulle  
z_t = -1 si le joueur au trait dans S_t perd
```

Le format d'une experience d'apprentissage devient donc:

```
(S_t, LegalMask_t, pi_MCTS_t, z_t)
```

Le reseau apprend alors simultanement:

```
P_theta(a | S) -> pi_MCTS  
V_theta(S)    -> z_terminal
```

4. Difference entre dataset teacher et dataset vivant

Aspect	Dataset teacher / Minimax	Dataset RL / self-play
Producteur des labels	Minimax, heuristiques ou teacher.	Network + MCTS + resultat terminal.
Cible Policy	best_action ou policy_target souvent quasi one-hot.	Distribution de visites π _MCTS.
Cible Value	wdl_target derive du teacher.	Resultat reel z de la partie.
Role scientifique	Baseline, audit, warm-start optionnel.	Source principale d'apprentissage autonome.
Nature du corpus	Fixe.	Evolutif, genere a chaque generation.

5. Warm-start supervise: option experimentale, pas dependance permanente

Deux options doivent etre comparees plutot que tranchees arbitrairement. La premiere consiste a apprendre entierement depuis un reseau aleatoire:

```
Random Network -> SelfPlay -> RL
```

C'est la solution la plus pure scientifiquement, mais les premieres parties peuvent etre faibles et l'apprentissage plus lent. La seconde consiste a utiliser temporairement Minimax pour initialiser le reseau:

```
Minimax -> Network_0 -> SelfPlay -> RL
```

Dans ce cas, le teacher sert uniquement de demarreur. Il ne devient pas le professeur permanent du modele. L'experience a mener est donc:

```
RL_scratch vs SupervisedWarmStart + RL
```

L'evaluation devra verifier si le warm-start accelere simplement l'apprentissage ou s'il enferme le modele dans les limites du teacher.

6. Responsabilites du moteur, du reseau et de la recherche

L'apprentissage repose sur une separation stricte de trois responsabilites. Le moteur Songo reste l'autorite deterministe du jeu: pour un etat S et une action legale a , il calcule exactement $S' = T(S, a)$. Il gere la legalite des coups, le semis, la circulation des graines, les captures, la transmission, la famine, le changement de joueur, les etats terminaux et le resultat final. Le reseau n'apprend donc pas les regles du Songo.

Le reseau neuronal produit deux informations: une Policy $P_{\theta}(a | S)$, qui represente sa preference pour les actions disponibles, et une Value $V_{\theta}(S)$, qui estime la qualite de la position pour le joueur au trait. La recherche, dont MCTS/PUCT est l'hypothese principale, utilise ces sorties et les transitions exactes du moteur pour explorer plusieurs continuations.

```
S -> Network(P, V) -> Recherche ->  $\pi$  -> Selection action -> Moteur ->  $S'$ 
```

La distribution π produite par la recherche devient ensuite une cible d'apprentissage pour la Policy du reseau.

7. Exploration pendant les premieres generations

Au debut, les preferences du reseau sont peu fiables. Il serait donc dangereux de toujours selectionner l'action ayant la plus forte probabilite selon un modele encore ignorant. Le systeme doit conserver une capacite d'exploration, notamment dans les premieres generations de self-play.

Dans une approche MCTS/PUCT, une solution consiste a perturber les probabilites a la racine par un bruit de Dirichlet:

```
 $P'(a) = (1 - \epsilon) * P_{\theta}(a | S) + \epsilon * \eta_a$   
 $\eta \sim \text{Dir}(\alpha)$ 
```

Cette perturbation n'a pas pour objectif d'ameliorer directement le coup joue. Elle evite que l'agent se limite trop rapidement aux actions actuellement preferees par un reseau encore mal calibre.

8. Amelioration de la Policy par la recherche

La Policy brute du reseau et la Policy issue de la recherche ne doivent pas etre confondues. Le reseau fournit d'abord une distribution initiale:

```
P_theta(S) = [0.20, 0.18, 0.17, 0.15, 0.12, 0.10, 0.08]
```

Apres plusieurs simulations, la recherche obtient des nombres de visites $N(S, a)$, par exemple:

```
N(S, .) = [12, 8, 41, 5, 79, 3, 2]
```

Ces visites sont converties en cible de Policy:

```
pi(a | S) = N(S, a)^(1 / tau) / sum_b N(S, b)^(1 / tau)
```

La recherche agit donc comme un mecanisme d'amelioration de politique:

```
P_theta -> MCTS/PUCT -> pi
```

9. Fonction de perte Policy, Value et variante WDL

Une fois les experiences generees, le reseau doit ajuster ses parametres theta pour rapprocher sa Policy de π_{MCTS} et sa Value du resultat terminal z :

```
p_theta(a | S) ~ pi(a | S)
v_theta(S) ~ z
```

La perte de Policy utilise une cross-entropy entre la distribution de recherche et la distribution predite:

```
L_policy = - sum_{a in A(S)} pi(a | S) * log p_theta(a | S)
```

Pour la Value, la baseline initiale est une valeur scalaire $v_{\text{theta}}(S)$ in $[-1, +1]$ entraine par MSE:

```
L_value = (z - v_theta(S))^2
```

Une variante a tester consiste a remplacer la Value scalaire par une tete WDL produisant trois probabilites:

```
P(W | S), P(D | S), P(L | S)
```

Dans ce cas, le resultat terminal devient une classe et la perte Value devient une cross-entropy:

```
L_WDL = - sum_{c in {W, D, L}} y_c * log p_theta(c | S)
```

La position methodologique retenue est donc:

```
Baseline: Value scalaire + MSE
Variante experimentale: WDL + Cross-Entropy
```

La perte globale avec Value scalaire est:

```
L(theta) = lambda_p * L_policy
          + lambda_v * L_value
          + lambda_r * ||theta||_2^2
```

Avec une tete WDL, le terme L_{value} est remplace par L_{WDL} . Les coefficients λ_p , λ_v , λ_w et λ_r sont des hyperparametres experimentaux.

10. Non-convexite, gradient et retropropagation

La convexite doit etre distinguee de la fonction de perte choisie. Une fonction simple comme $f(x) = x^2$ est convexe, mais la perte du reseau Songo depend d'une succession de transformations neuronales: encodage, propagation relationnelle, pooling, Policy, Value et loss. La fonction $L(\theta)$, consideree par rapport a l'ensemble des parametres du reseau profond, est donc generalement non convexe.

Apres le calcul de la perte, le systeme calcule le gradient:

```
grad_theta L
```

La mise a jour conceptuelle est:

```
theta_{t+1} = theta_t - eta * grad_theta L
```

En pratique, la retropropagation calcule les gradients et un optimiseur comme AdamW peut servir de solution initiale.

```
Forward Pass -> Loss -> Backpropagation -> Gradients -> Optimiseur -> Mise a jour des poids
```

11. Deux niveaux d'amelioration complementaires

L'architecture d'apprentissage combine deux mecanismes. Le premier ameliore la decision locale: a partir de P_{θ} , la recherche produit une distribution π plus informee. Le second ameliore le reseau: a partir des exemples (S, π, z) , l'optimisation met a jour les parametres.

```
P_theta -> MCTS/PUCT -> pi
theta   -> optimisation -> theta'
```

La boucle complete est donc:

```
Reseau
-> Recherche
-> Meilleures decisions
-> Parties
-> Experiences
-> Optimisation
-> Meilleur reseau
```

12. Phases

Phase	Objectif	Sortie attendue
0. Validation moteur	Verifier regles, conservation des 70 graines, famine, captures, transmission.	Moteur fiable.
1. Baseline MLP	Obtenir un pipeline complet simple.	Reference minimale.
2A. RL depuis zero	Tester l'apprentissage autonome sans teacher.	Reference scientifique pure.
2B. Warm-start supervise optionnel	Amorcer la politique sans conserver la dependance au teacher.	SRN initialise a comparer a 2A.
3. Self-play + MCTS	Generer le vrai corpus d'apprentissage.	Replay buffer vivant et generations.
4. Population d'adversaires	Eviter l'auto-exploitation.	Robustesse contre anciennes versions.
5. Curriculum	Travailler les situations critiques.	Progres sur cas difficiles.
6. Ablations	Verifier que les relations SRN apportent un gain.	Validation scientifique.

13. Boucle evolutive

```
checkpoint_k
-> self-play avec MCTS
-> replay buffer
-> entraînement de checkpoint_{k+1}
-> evaluation contre baseline, Minimax et anciens checkpoints
-> promotion ou rejet
-> nouvelle generation
```

Le cycle conceptuel complet est:

```
Network
-> MCTS
-> SelfPlay
-> Dataset (S, pi, z)
-> Training
-> New Network

Algorithme ENTRAINEMENT_SONGO_RL
1. Initialiser le reseau N_0.
2. Initialiser le Replay Buffer D.
3. Pour chaque generation g:
    SELF-PLAY:
        pour plusieurs parties:
            initialiser une position S
```

```

    tant que S n'est pas terminal:
        legal_mask = moteur.legal_actions(S)
        P, V = Network(S)
        pi = MCTS_PUCT(S, P, V, moteur)
        action = selection(pi, temperature)
        enregistrer (S, legal_mask, pi)
        S = moteur.play(S, action)
        z = moteur.result(S)
    convertir z pour chaque position de la trajectoire
    ajouter (S_t, legal_mask_t, pi_t, z_t) au buffer
TRAINING:
    pour plusieurs etapes:
        batch = ReplayBuffer.sample()
        p_theta, v_theta = Network(batch.S)
        calculer L_policy, L_value ou L_WDL, regularisation
        backpropagation
        optimizer.step()
EVALUATION:
    evaluer contre champion, anciens checkpoints, Minimax et D_LAB
PROMOTION:
    promouvoir seulement si les criteres experimentaux sont satisfaits

```

14. Promotion d'un checkpoint

Un checkpoint ne doit pas être promu uniquement parce que sa loss baisse. Il doit progresser en match, rester calibré en Value, conserver une Policy suffisamment diverse et ne pas regresser sur les positions critiques.

15. Algorithme de self-play

```

Algorithme GenerateSelfPlayGames(network_k, num_games)
Sortie:
    experiences: liste de positions d'entrainement

1. experiences = []
2. pour game_id dans 1..num_games:
    S = position_initiale_ou_position_curriculum()
    trajectory = []
    tant que S non terminal:
        legal_mask = moteur.legal_actions(S)
        pi_mcts = MCTS_Search(S, network_k)
        temperature = schedule_temperature(move_number)
        action = sample(pi_mcts, temperature)
        trajectory.append((S, legal_mask, pi_mcts, joueur_au_trait))
        S = moteur.play(S, action)
    z_final = moteur.result(S)
    pour chaque position t dans trajectory:
        z_t = z_final du point de vue du joueur_au_trait_t
        experiences.append((S_t, legal_mask_t, pi_mcts_t, z_t))
3. retourner experiences

```

16. Algorithme d'entrainement d'une generation

```

Algorithme TrainGeneration(network_k, replay_buffer)
Sortie:
    candidate network_{k+1}

1. copier network_k vers candidate
2. pour epoch dans 1..E:
    pour batch dans replay_buffer.sample_batches():
        G = BuildGraph(batch.states)
        pi_pred, v_pred = candidate(G, batch.legal_masks)
        loss_policy = CE(batch.pi_mcts, pi_pred)
        loss_value = MSE(batch.z, v_pred)
        loss_aux = optional_auxiliary_losses(batch)
        loss = lambda_p * loss_policy
            + lambda_v * loss_value
            + lambda_aux * loss_aux
            + weight_decay
        backprop(loss)
        optimizer.step()
3. retourner candidate

```

17. Algorithme de promotion

```
Algorithme PromoteOrReject(candidate, champion)
1. evaluer candidate contre champion sur N parties
2. evaluer candidate contre anciens checkpoints
3. evaluer candidate contre baseline Minimax/SongoFish
4. evaluer candidate sur positions critiques
5. mesurer:
    win_rate, Elo, calibration Value, entropie Policy,
    regressions tactiques, cout inference
6. si candidate depasse les seuils minimaux:
    promouvoir candidate comme nouveau champion
sinon:
    conserver champion et archiver candidate comme experience
```

XI. Dataset d'apprentissage et contrat d'entrainement

1. Principe du contrat de donnees

Le corpus principal d'entrainement n'est pas constitue manuellement et n'est pas annote par Minimax. Il est produit automatiquement pendant les parties de self-play. Pour chaque position effectivement rencontree, le systeme enregistre un exemple:

```
E_t = (S_t, M_t, pi_t, z_t)
```

Ces quatre elements n'ont pas le meme statut. S_t est l'entree du reseau. M_t est une contrainte de validite des actions. pi_t est le label de la tete Policy. z_t est le label de la tete Value. Il faut donc eviter de parler indistinctement de tous les champs comme de labels.

```
S_t -> Policy cible: pi_t
S_t -> Value cible: z_t
```

2. Structure conceptuelle d'un exemple

```
TrainingExample
+-- state
+-- legal_mask
+-- policy_target
+-- value_target
+-- metadata
```

Les quatre premiers champs participent directement ou indirectement a l'entrainement. metadata sert a l'analyse, au debogage et a la reproductibilite, mais ne constitue pas une entree strategique du reseau.

3. state: etat d'entree du reseau

Le champ state contient l'etat necessaire pour reconstruire exactement la position donnee au reseau:

```
S_t = (Board_t, Stores_t, Player_t)

board = [5, 5, 3, 8, 0, 4, 6, 2, 7, 5, 1, 9, 4, 3]
stores = [4, 4]
player = 1
```

Le format reel devra rester coherent avec la representation canonique retenue apres verification de la symetrie du moteur. Le point important est que l'etat brut soit conserve dans le dataset. Les caracteristiques derivees du reseau doivent etre calculees par l'encodeur:

```
S_t -> Encoder -> X_t
```

Cela evite de figer definitivement le dataset sur une version particuliere de l'architecture neuronale.

4. legal_mask: contrainte de validite, pas label

Le Songo possede au maximum sept actions locales pour le joueur courant. Le moteur fournit donc un masque:

```
M_t in {0, 1}^7
M_t = [1, 1, 0, 1, 1, 0, 1]
```

Cet exemple signifie que les actions 0, 1, 3, 4 et 6 sont legales, tandis que 2 et 5 sont interdites. Le legal mask n'est pas un label a apprendre: le reseau n'a pas pour objectif de predire la legalite. Il sert a imposer une probabilite nulle aux coups interdits.

```
logit'_a = logit_a si M_a = 1
logit'_a = -inf si M_a = 0

p_theta(a | S) = Softmax(logit')
```

5. policy_target: label de la tete Policy

Le champ `policy_target`, note `pi_t`, contient sept valeurs, une par action locale. Il ne contient pas le `best_action` de Minimax. Il contient la distribution issue des visites de MCTS:

```
pi_t(a) = N(S_t, a)^(1 / tau) / sum_b N(S_t, b)^(1 / tau)
```

Par exemple:

```
policy_target = [0.03, 0.07, 0.00, 0.55, 0.24, 0.00, 0.11]
```

La somme des probabilités vaut 1 et les actions illégales ont une probabilité nulle. Cette distribution est plus riche qu'un simple one-hot `best_action = 3`, car elle conserve l'information que d'autres coups ont aussi reçu de l'attention pendant la recherche.

6. value_target: label de la tete Value

Le champ `value_target`, note `z_t`, est le second label. Dans la baseline Value scalaire:

```
z_t = +1 victoire du joueur represente dans S_t
z_t = 0 partie nulle
z_t = -1 defaite du joueur represente dans S_t
```

Ce label ne provient ni de Minimax, ni de MCTS, ni de la Value du réseau. Il provient du résultat réel de la partie.

```
pi_t = produit par MCTS
z_t = produit par le resultat terminal
```

7. Perspective du joueur

La perspective du joueur est critique. Si la partie est gagnée par le joueur 1, une position représentée du point de vue du joueur 1 reçoit `z = +1`, tandis qu'une position représentée du point de vue du joueur 2 reçoit `z = -1`. Si le réseau voit toujours la position du point de vue du joueur au trait, alors la cible doit être calculée dans cette même perspective.

```
z_t = +1 si winner = playerPerspective_t
z_t = 0 si draw
z_t = -1 si winner != playerPerspective_t
```

Une erreur sur cette conversion rendrait la tête Value incohérente et pourrait compromettre l'apprentissage.

8. Finalisation a la fin de la partie

Pendant la partie, le système connaît immédiatement `S_t`, `M_t` et `pi_t`, mais il ne connaît pas encore `z_t`. Il doit donc conserver temporairement:

```
(state, legal_mask, policy_target, player_perspective)
```

Lorsque le résultat terminal est connu, le système revient sur la trajectoire et complète chaque exemple avec son `value_target`.

```
(S_0, M_0, pi_0, ?) -> (S_0, M_0, pi_0, z_0)
(S_1, M_1, pi_1, ?) -> (S_1, M_1, pi_1, z_1)
```

9. Variante WDL sans regenerer les parties

Si une tête WDL est testée, il n'est pas nécessaire de régénérer les parties si le résultat terminal et la perspective ont été conservés correctement. Le scalaire peut être transformé en classe:

```
z = +1 -> y = [1, 0, 0]
z = 0 -> y = [0, 1, 0]
z = -1 -> y = [0, 0, 1]
```

10. Metadonnees conservees hors entree reseau

Les métadonnées servent au diagnostic, à l'audit et à la reproductibilité. Elles ne doivent pas automatiquement devenir des caractéristiques d'entrée du réseau.

```
metadata:
```

```

game_id
ply
generation
checkpoint_id
player
winner
final_score
mcts_simulations
temperature
root_visit_counts
root_value

```

11. Format conceptuel d'un enregistrement

```

{
  "state": {
    "board": [5, 5, 3, 8, 0, 4, 6, 2, 7, 5, 1, 9, 4, 3],
    "stores": [4, 4],
    "player": 1
  },
  "legal_mask": [1, 1, 0, 1, 1, 0, 1],
  "policy_target": [0.03, 0.07, 0.00, 0.55, 0.24, 0.00, 0.11],
  "value_target": 1,
  "metadata": {
    "game_id": 15284,
    "ply": 27,
    "generation": 12,
    "checkpoint_id": "songo_g12",
    "mcts_simulations": 400,
    "temperature": 1.0
  }
}

```

Ces nombres illustrent un format conceptuel, pas des parametres definitifs. Le format physique de stockage, JSONL, NumPy, Parquet ou shards binaires, reste une decision d'implementation.

12. Ce qui ne doit pas etre label principal

Le dataset RL principal n'utilise pas comme labels principaux:

```

best_action_minimax
minimax_score
teacher_value
bidoua_score
yinda_score
heuristic_score
safe_territory_score

```

Ces informations peuvent exister dans un corpus experimental separe et servir a analyser ce que le reseau apprend. Le contrat d'apprentissage principal reste:

```

LABELS = {pi_MCTS, z_terminal}

```

13. Passage du dataset au reseau

Pour un mini-batch $B = \{(S_i, M_i, pi_i, z_i)\}$, le reseau recoit S_i et le masque M_i . Il produit des logits de politique et une valeur:

```

logits_i, v_i = Network(S_i)
p_i = Softmax(mask(logits_i, M_i))

comparer p_i a pi_i
comparer v_i a z_i

```

C'est le coeur du code d'entrainement. Le reseau apprend la distribution de recherche pi_MCTS , pas seulement l'action effectivement jouee.

14. Train step conceptuel

```
def train_step(model, optimizer, batch, device):
    states = batch["state"].to(device)
    legal_mask = batch["legal_mask"].to(device)
    policy_target = batch["policy_target"].to(device)
    value_target = batch["value_target"].to(device)

    policy_logits, value_pred = model(states)
    masked_logits = policy_logits.masked_fill(
        ~legal_mask.bool(),
        float("-inf")
    )
    log_policy = torch.log_softmax(masked_logits, dim=-1)
    policy_loss = -(policy_target * log_policy).sum(dim=-1).mean()

    value_pred = value_pred.squeeze(-1)
    value_loss = F.mse_loss(value_pred, value_target)

    loss = lambda_policy * policy_loss + lambda_value * value_loss
    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=grad_clip)
    optimizer.step()

    return {
        "loss": loss.item(),
        "policy_loss": policy_loss.item(),
        "value_loss": value_loss.item()
    }
```

15. Ne pas entrainer sur le seul coup joue

Si $\pi = [0.05, 0.05, 0, 0.55, 0.30, 0, 0.05]$ et que l'action 3 est tirée, stocker seulement $\text{action_played} = 3$ transformerait la cible en one-hot et ferait perdre l'information que l'action 4 avait obtenu 30 % des visites. La Policy doit donc apprendre la distribution de recherche, pas seulement l'action échantillonnée.

16. Metriques a enregistrer pendant l'entrainement

La simple loss ne suffit pas. A chaque periode d'entrainement, il faut suivre au minimum:

```
generation
training_step
loss_total
loss_policy
loss_value
policy_entropy
value_mae
gradient_norm
learning_rate
```

Ces metriques permettent de distinguer un reseau qui n'apprend pas d'un reseau qui apprend le replay buffer mais ne devient pas meilleur joueur.

17. Loss faible et force strategique

Une loss plus faible ne signifie pas necessairement une meilleure IA. La loss mesure surtout la capacite du reseau a approximer (π, z) sur le replay buffer. La force reelle doit etre mesuree dans le jeu et sur les tests strategiques.

```
Performance d'apprentissage: losses et calibration
Performance strategique: parties, tournois, positions critiques
```

18. Contrat definitif self-play vers training

```
Self-play produit:
S, LegalMask, pi_MCTS, Result

Dataset transforme en:
Input      = S
Constraint  = LegalMask
PolicyLabel = pi_MCTS
ValueLabel  = z_terminal

Network produit:
PolicyPrediction = p_theta
ValuePrediction  = v_theta

Training minimise:
L_policy = - sum_a pi_a * log p_a
L_value  = (z - v)^2
L         = lambda_p * L_policy
          + lambda_v * L_value
          + lambda_r * R(theta)
```

19. Statut du dataset v001 10k

L'audit du dataset v001 montre qu'il contient 10 000 positions, separees en 7 979 train, 908 validation et 1 113 test. Les positions gardent 70 graines et les trajectoires sont separees entre splits. C'est une bonne base de laboratoire.

Mais ce dataset ne doit pas etre considere comme le corpus d'apprentissage final du modele autonome. Il contient surtout des decisions de teacher, avec une part faible de positions exactes, des policy_target souvent one-hot, et des action_values qui ne couvrent pas toujours tous les coups legaux.

Decision documentaire. Le corpus de 10 000 positions est un laboratoire de positions reelles pour l'analyse, le test et l'evaluation. Il ne constitue pas le corpus d'entrainement du futur modele autonome. Le corpus principal du modele devra etre produit par self-play, MCTS et resultats terminaux reels.

On distinguera donc explicitement deux ensembles:

```
D_LAB = corpus fixe de positions reelles pour analyse, tests et evaluation
D_RL  = corpus vivant produit par self-play: (S, LegalMask, pi_MCTS, z)
```

D_LAB peut servir a verifier que le reseau accepte correctement les etats, comparer ses decisions avec celles de Minimax, rechercher des positions interessantes, mesurer les changements entre generations et construire une batterie de tests strategiques. Il ne doit pas etre relabellise artificiellement pour devenir le dataset principal d'entrainement autonome. D_RL, lui, est produit par l'agent et evolue en meme temps que son intelligence.

20. Informations utiles du dataset existant

Champ	Utilite	Limite
state	Etat canonique exploitable.	Ne contient pas directement ZT/TS/TR/TM.
legal_mask	Masque action fiable.	Produit par le moteur, pas a apprendre.
best_action	Reference Minimax et comparaison.	Ne doit pas devenir la verite strategique finale.
policy_target	Audit du comportement teacher; warm-start optionnel.	Souvent quasi one-hot, moins riche que pi_MCTS.
wdl_target	Signal de laboratoire ou baseline.	Derive de l'evaluation du teacher, pas de resultats reels.
consequences	Capture, mobilite, famine.	Pas Bidoua/Yinda/Transport explicites.

XI. Dataset d'apprentissage et contrat d'entrainement - suite

21. Replay buffer cible D_RL

Experience minimale :

```
state
legal_mask
policy_target = pi_mcts
value_target = z_terminal
metadata
```

Metadonnees recommandees :

```
game_id, ply, generation_id, checkpoint_id, winner, final_score,
mcts_budget, temperature, root_visit_counts (optionnel)
```

Les tags strategiques eventuels servent a l'analyse et au curriculum ; ils ne sont pas des labels principaux.

22. Algorithme d'insertion

Algorithme ReplayBuffer_Insert(experiences)

1. pour chaque experience e :
 - verifier la validite de l'etat
 - verifier que legal_mask correspond au moteur
 - verifier $\pi_{\text{mcts}}[a] = 0$ pour les actions illegales
 - verifier $\text{somme}(\pi_{\text{mcts}}) \approx 1$ sur les actions legales
 - verifier z_{terminal} dans $\{-1, 0, +1\}$
 - verifier la perspective utilisee pour z
 - ajouter e au buffer
2. si $\text{taille}(\text{buffer}) > \text{capacite}$:
 - appliquer la politique de retention configuree

La deduplication n'est pas imposee par default : deux occurrences d'un meme etat peuvent provenir de recherches differentes et porter des distributions π_{MCTS} differentes. Toute agregation devra etre explicite et testee.

23. Echantillonnage des mini-batches

BASELINE RETENUE :

echantillonnage uniforme dans une fenetre de replay contenant des experiences recentes et suffisamment variees.

Cette baseline est volontairement simple. Elle permet d'evaluer le pipeline sans introduire une politique d'echantillonnage qui modifierait elle-meme la distribution d'apprentissage.

VARIANTES EXPERIMENTALES ULTERIEURES :

- priorisation par erreur / surprise ;
- quotas de generations historiques ;
- sur-echantillonnage de positions rares ou critiques ;
- melange recent / historique.

Les proportions 70 % / 20 % / 10 % de la version precedente ne sont donc plus prescrites. Elles peuvent devenir une experience, mais ne constituent pas le contrat de base.

Algorithme SampleBatchUniform(buffer, batch_size)

1. definir la fenetre active du replay buffer
2. tirer batch_size experiences uniformement
3. verifier la diversite minimale du batch si necessaire
4. retourner le batch

XII. Reward, nulles, exploration et curriculum

1. Reward terminal

Le signal principal est terminal :

- +1 victoire
- 0 partie nulle
- 1 defaite

La cible est toujours exprimee dans la meme perspective que l'etat donne au reseau.

2. Parties nulles

Les nulles restent des resultats distincts. La baseline Value scalaire utilise $z=0$.

Une variante WDL peut modeliser explicitement Win / Draw / Loss.

3. Exploration

Mecanismes candidats :

- temperature sur la distribution de visites MCTS ;
- bruit de Dirichlet a la racine pendant le self-play ;
- diversite de checkpoints adverses si le protocole l'utilise.

Les parametres exacts sont a calibrer.

4. Curriculum

Le livre mobilise dans le projet invite a ne pas imposer une decomposition classique ouverture / milieu / fin comme cadre strategique principal. Le curriculum ne doit donc pas dependre d'une "phase_proxy" supposee.

Un curriculum eventuel peut plutot etre construit a partir de difficultes observables et mesurables :

- nombre de coups legaux ;
- longueur de circulation ;
- transfert, retour, transferts multiples ;
- famine et transmission ;
- positions critiques Bidoua / Yinda / Transport-Bidoua ;
- situations ou l'effet utile d'un coup est differe.

5. Pourquoi eviter un reward shaping trop fort ?

Recompenser directement capture, mobilite ou Bidoua risquerait de reintroduire l'heuristique humaine dans la fonction objectif. La version principale conserve donc le resultat terminal comme signal strategique fondamental.

6. Temperature

La temperature doit etre definie comme une politique d'exploration, non comme une fonction d'une phase strategique predefinie.

Algorithme Temperature(ply, training_config)

1. si self_play = faux :
 - retourner tau_inference
2. lire la schedule de temperature configuree
3. calculer tau selon le nombre de coups joues (ply) ou une schedule explicite
4. retourner tau

Une variante adaptative basee sur l'entropie de la recherche pourra etre testee ulterieurement.

2. Arborescence réelle observée

```
songo/
+-- packages/songo_ai/
|   +-- songo/
|   |   +-- rules.py
|   |   +-- fast_rules.py
|   +-- search/
|   |   +-- negamax.py
|   +-- teachers/
|   |   +-- deep_teacher.py
|   |   +-- cache.py
|   +-- dataset/
|   |   +-- build.py
|   |   +-- schema.py
|   |   +-- validate.py
|   |   +-- merge.py
|   |   +-- external_import.py
|   +-- generation/
|   |   +-- trajectories.py
|   |   +-- sampling.py
|   |   +-- agents.py
|   +-- model/
|   |   +-- features.py
|   |   +-- dataset.py
|   |   +-- network.py
|   |   +-- losses.py
|   |   +-- train.py
|   |   +-- inference.py
|   |   +-- registry.py
|   +-- hybrid/
|   |   +-- network_eval.py
|   |   +-- songofish.py
|   +-- evaluation/
|   |   +-- tournament.py
|   +-- cloud/
|   |   +-- config.py
|   |   +-- jobs.py
|   |   +-- providers.py
|   |   +-- storage.py
+-- apps/
|   +-- trainer/
|   +-- table/
+-- data/
|   +-- dataset_v001_10k/
|   +-- dataset_v002_100k/
|   +-- dataset_v003_110k/
|   +-- dataset_v004_290k/
|   +-- dataset_v005_400k/
|   +-- checkpoints/
+-- docs/
```

3. Roles des modules existants

Module existant	Rôle actuel	Rôle dans l'architecture SRN/RL
songo/rules.py	Moteur de regles de reference, etat, coups legaux, transitions, terminal.	Reference fonctionnelle a conserver pour les tests et la verite des regles.
songo/fast_rules.py	Version rapide/Numba du moteur, legal_mask, play_local, clone_for_search.	Base privilegiee pour MCTS et self-play intensif.
search/negamax.py	Recherche Alpha-Beta/Negamax et evaluation heuristique.	Baseline, adversaire, teacher optionnel, pas source principale RL.
teachers/deep_teacher.py	Annotation des positions par teacher profond.	Warm-start et audit, mais pas label principal du dataset RL.
dataset/*	Construction, schema, validation et fusion des datasets teacher.	A etendre avec un schema RL pour (S, M, pi, z) sans casser les datasets existants.
generation/*	Trajectoires, sampling, agents pour produire positions et datasets.	A reutiliser pour self-play, en remplaçant les labels teacher par pi_MCTS et z_terminal.
model/features.py	Features vectorielles MLP: 33 entrees derivees.	Baseline MLP et comparaison; ne doit pas figer l'encodeur SRN.
model/network.py	SongoNet MLP residuel avec tetes Policy, WDL et Q-actions.	Baseline actuelle; SRN doit etre ajoute comme nouvelle architecture parallele.
model/dataset.py	ObservationDataset charge les shards JSONL teacher.	A completer par un SelfPlayDataset respectant le contrat (state, legal_mask, policy_target, value_target).
model/train.py	Boucle d'entrainement supervise, checkpoints, validation.	A reutiliser et generaliser pour trainer MLP teacher et SRN self-play.
model/registry.py	Registry, champion, versions, promotion manuelle.	A conserver pour les generations RL et la selection de champions.
hybrid/*	Utilisation reseau + recherche SongoFish.	Pont utile pour integrer un reseau appris dans un agent de jeu.
evaluation/tournament.py	Matches, evaluation, comparaison d'agents.	Base pour evaluer checkpoints RL contre champion, anciens modeles et Minimax.
cloud/*	Execution locale/GCP, stockage, jobs.	Infrastructure utile pour generer self-play et entrainer a plus grande echelle.
apps/trainer	Scripts et notebooks de construction/entrainement/export.	Point d'entree experimental a adapter avant industrialisation.
apps/table	Interface de jeu locale avec controleurs.	Support pour tester les agents et l'integration utilisateur.

4. Architecture actuelle du modele

Le modele existant principal est SongoNet dans `model/network.py`. Il s'agit d'un MLP residuel leger: une couche d'entree, plusieurs blocs residuels, puis trois tetes:

```
SongoNet actuel:
input: observation_features(state, legal_mask) -> 33 valeurs
trunk: MLP residuel
outputs:
  policy_head -> 7 logits
  wdl_head    -> 3 logits
  q_head      -> 7 valeurs
```

Cette architecture est utile comme baseline. Elle ne doit pas etre supprimee. Elle permettra de comparer la valeur reelle du SRN contre un modele vectoriel deja present dans le code.

5. Architecture cible comme extension du depot existant

La cible ne doit pas remplacer brutalement `songo_ai`. Elle doit ajouter des modules manquants tout en gardant les responsabilites existantes. Une adaptation raisonnable est:

```
packages/songo_ai/
+-- songo/                # conserve: regles exactes et moteur rapide
+-- search/
|   +-- negamax.py        # conserve: baseline
|   +-- mcts.py           # nouveau: MCTS/PUCT
|   +-- puct.py           # nouveau: formule de selection
+-- model/
|   +-- network.py        # conserve: MLP baseline
|   +-- srn_graph.py      # nouveau: construction graphe 14 noeuds
|   +-- srn_network.py    # nouveau: SRN Policy/Value
|   +-- selfplay_dataset.py # nouveau: contrat (S, M, pi, z)
|   +-- losses.py         # etendre: policy CE, scalar value, WDL
|   +-- train.py          # etendre: trainer commun
+-- generation/
|   +-- trajectories.py    # conserve
|   +-- selfplay.py       # nouveau: parties Network+MCTS
+-- evaluation/
|   +-- tournament.py     # conserve et etendre
|   +-- critical_positions.py # nouveau
+-- model/registry.py     # conserve: generations et champions
```

6. Regle de migration

La migration doit suivre une regle simple: reutiliser les modules existants lorsqu'ils correspondent deja au contrat cible, et ajouter un module seulement lorsqu'une responsabilite nouvelle apparait. Il ne faut pas dupliquer le moteur de regles, le systeme de registry, l'evaluation par tournoi ou les scripts de generation si une extension suffit.

Besoin SRN/RL	Action recommandee
Moteur exact	Reutiliser <code>songo/rules.py</code> et <code>fast_rules.py</code> .
MCTS/PUCT	Ajouter <code>search/mcts.py</code> sans modifier <code>Negamax</code> .
Graphe SRN	Ajouter <code>model/srn_graph.py</code> et tester contre le moteur.
Reseau SRN	Ajouter <code>model/srn_network.py</code> en gardant <code>network.py</code> comme baseline.
Dataset RL	Ajouter <code>model/selfplay_dataset.py</code> ou <code>dataset/selfplay_schema.py</code> .
Self-play	Ajouter <code>generation/selfplay.py</code> en reutilisant <code>agents/trajectoires</code> si possible.
Training	Etendre <code>model/train.py</code> pour accepter MLP teacher et SRN RL.

Besoin SRN/RL	Action recommandee
Evaluation	Etendre <code>evaluation/tournament.py</code> et ajouter positions critiques.
Checkpoints	Conserver <code>model/registry.py</code> .

7. Ordre d'adaptation recommande

1. Conserver et tester le moteur existant:
`rules.py` + `fast_rules.py`.
2. Ajouter le contrat de donnees self-play:
(`state`, `legal_mask`, `policy_target`, `value_target`, `metadata`).
3. Ajouter `GraphBuilder/SRN` sans toucher au MLP existant.
4. Ajouter MCTS/PUCT en reutilisant `fast_rules.py`.
5. Produire un petit self-play dataset de test.
6. Adapter `train.py` pour entrainer SRN sur le contrat (`S`, `M`, `pi`, `z`).
7. Evaluer via `tournament.py` et `registry.py`.
8. Comparer MLP baseline, SRN sans MCTS, SRN + MCTS.

Decision d'architecture logicielle. Le projet doit evoluer par extension du package existant `songo_ai`. La documentation ne prescrit donc pas une arborescence neuve a recopier; elle fixe les responsabilites a ajouter autour des modules deja presents.

XIV. Squelette de code

1. Etat et graphe

```
@dataclass(frozen=True)
class SongoState:
    pits: Tuple[int, ...]          # 14 cases jouables
    stores: Tuple[int, int]        # magasins physiques ou relatifs selon representation validee
    turn: int                      # conserve tant que la canonicalisation n'est pas formellement validee
    legal_mask: Tuple[bool, ...]

@dataclass
class SongoGraph:
    node_features: Tensor          # [14, node_dim]
    global_features: Tensor        # [global_dim]
    edges_by_relation: Dict[str, Tensor]

class SongoGraphBuilder:
    def build(self, state, relation_config):
        node_features = self.encode_nodes_minimal(state)
        global_features = self.encode_global_minimal(state)
        edges = {
            "next": self.next_edges(),
            "prev": self.prev_edges(),
        }
        if relation_config.mirror:
            edges["mirror"] = self.mirror_edges()
        if relation_config.dynamic_sowing:
            edges["dynamic_sowing"] = self.dynamic_sowing_edges(state)
        return SongoGraph(node_features, global_features, edges)
```

2. Bloc relationnel

```
class RelationalBlock(nn.Module):
    def __init__(self, dim, relations):
        super().__init__()
        self.messages = nn.ModuleDict({
            r: nn.Sequential(
                nn.Linear(2 * dim, dim), nn.ReLU(), nn.Linear(dim, dim)
            ) for r in relations
        })
        self.update = nn.Sequential(
            nn.Linear(2 * dim, dim), nn.ReLU(), nn.Linear(dim, dim)
        )
        self.norm = nn.LayerNorm(dim)

    def forward(self, h, edges_by_relation):
        agg = torch.zeros_like(h)
        for rel, edges in edges_by_relation.items():
            src, dst = edges
            pair = torch.cat([h[src], h[dst]], dim=-1)
            msg = self.messages[rel](pair)
            agg.index_add_(0, dst, msg)
        out = self.update(torch.cat([h, agg], dim=-1))
        return self.norm(h + out)
```

3. Policy-Value SRN : Policy action-conditionnee

```
class SongoRelationalNetwork(nn.Module):
    def __init__(self, node_dim, global_dim, hidden_dim=64, blocks=3, relations=None):
        super().__init__()
        relations = relations or ["next", "prev"]
        self.node_encoder = nn.Linear(node_dim, hidden_dim)
        self.global_encoder = nn.Linear(global_dim, hidden_dim)
        self.blocks = nn.ModuleList([
            RelationalBlock(hidden_dim, relations) for _ in range(blocks)
        ])

        # UN score par action, avec poids partages entre les 7 cases.
        self.policy_action = nn.Sequential(
            nn.Linear(3 * hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )
        self.value = nn.Sequential(
            nn.Linear(2 * hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
            nn.Tanh(),
        )

    def forward(self, graph, legal_mask):
        h = self.node_encoder(graph.node_features)
        g = self.global_encoder(graph.global_features)

        for block in self.blocks:
            h = block(h, graph.edges_by_relation)

        pooled = h.mean(dim=0)          # baseline ; attention pooling a comparer
        h_state = torch.cat([pooled, g], dim=-1)

        logits = []
        for a in range(7):
            action_input = torch.cat([h[a], h_state], dim=-1)
            logits.append(self.policy_action(action_input))
        logits = torch.stack(logits).squeeze(-1)
        logits = logits.masked_fill(~legal_mask.bool(), -1e9)

        value = self.value(h_state).squeeze(-1)
        return logits, value
```

XIV. Squelette de code - suite

4. MCTS

```
class MCTS:
    def search(self, state):
        root = Node(state)
        priors, value = self.evaluate(state)
        root.expand(priors)

        for _ in range(self.simulations):
            path = self.select(root)
            leaf = path[-1]

            if self.env.is_terminal(leaf.state):
                z = self.env.terminal_value(leaf.state)
            else:
                priors, z = self.evaluate(leaf.state)
                leaf.expand(priors)

            self.backup(path, z)

        return root.visit_distribution()
```

Le backup doit respecter la convention de perspective. Le signe ne doit être inverse que lorsque la perspective entre deux niveaux change effectivement selon le contrat d'état retenu.

5. Loss d'entraînement

```
def srn_loss(batch, model, weights):
    graphs = graph_builder.build_batch(batch.states)
    policy_logits, values = model(graphs, batch.legal_masks)

    masked_logits = policy_logits.masked_fill(
        ~batch.legal_masks.bool(), -1e9
    )
    log_probs = torch.log_softmax(masked_logits, dim=-1)

    policy_loss = -(batch.policy_target * log_probs).sum(dim=-1).mean()
    value_loss = F.mse_loss(values, batch.value_target)

    total = (
        weights.policy * policy_loss
        + weights.value * value_loss
    )
    return total, {
        "policy_loss": policy_loss.detach(),
        "value_loss": value_loss.detach(),
    }
```

La régularisation peut être portée par AdamW/weight_decay ou ajoutée explicitement, mais elle ne doit pas être comptée deux fois sans intention.

6. Boucle d'entraînement

```
def train_step(model, optimizer, batch):
    model.train()
    optimizer.zero_grad(set_to_none=True)

    loss, metrics = srn_loss(batch, model, weights)
    loss.backward()

    grad_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=grad_clip
    )
    optimizer.step()

    metrics["loss"] = loss.detach()
    metrics["grad_norm"] = grad_norm
    return metrics
```

7. Acteur de self-play : contrat (S, M, pi, z)

```
def run_selfplay_game(model, env, mcts, replay_buffer):
    trajectory = []
    state = env.new_game()

    while not env.is_terminal(state):
        legal_mask = env.legal_mask(state)
        pi_mcts = mcts.search(state)

        action = select_action(pi_mcts, temperature_schedule(state))

        trajectory.append({
            "state": state,
            "legal_mask": legal_mask,
            "policy_target": pi_mcts,
            "perspective": env.current_player(state),
        })

        state = env.play(state, action)

    result = env.result(state)

    for step in trajectory:
        z = result_from_player_view(result, step["perspective"])
        replay_buffer.add(
            state=step["state"],
            legal_mask=step["legal_mask"],
            policy_target=step["policy_target"],
            value_target=z,
        )
```

XV. Protocole experimental

1. Baselines

Modele	But	Question
MLP actuel	Reference simple sur features vectorielles.	Le SRN fait-il mieux qu'une bonne baseline?
SRN sans MCTS	Tester l'architecture seule.	Le graphe ameliore-t-il policy/value?
SRN + MCTS faible budget	Tester planification legere.	Quel gain par simulation?
SRN + MCTS budget croissant	Evaluer la scalabilite.	Le modele utilise-t-il mieux la recherche?
Minimax/Negamax	Adversaire et reference.	Le modele depasse-t-il progressivement le teacher?

2. Ablations

- supprimer la relation `dynamic_sowing`;
- supprimer `mirror` ou `border`;
- reduire le nombre de blocs;
- remplacer `pooling` moyen par `pooling` attentionnel;
- entrainer sans tetes auxiliaires;
- comparer policy one-hot teacher vs `pi_MCTS`;
- comparer avec et sans positions critiques dans le curriculum.

3. Mesures

Les mesures principales sont le taux de victoire, un Elo interne, la calibration de $V(S)$, l'entropie de la politique, la robustesse contre checkpoints historiques, la performance sur positions critiques et l'analyse des representations.

4. Algorithme d'ablation

```
Algorithme RunAblation(base_config, component)
1. entrainer modele_base avec base_config
2. creer config_ablation en retirant component
3. entrainer modele_ablation avec meme budget
4. comparer:
    loss validation
    matchs tete-a-tete
    positions critiques
    calibration Value
    cout inference
5. conclure:
    si ablation similaire a base:
        component non justifie ou mal exploite
    si ablation moins bonne que base:
        component utile
    si ablation meilleure que base:
        component nuisible ou sur-parametrant
```

XVI. Validation de l'intelligence strategique

Le modele ne doit pas seulement gagner en moyenne. Il doit montrer qu'il apprend les phenomenes caracteristiques du Songo.

Phenomenes	Test recommande	Signe attendu
Circulation longue	Positions ou une case traverse plusieurs zones.	Le modele anticipe l'arrivee utile.
Retour	Sequences avec retour sur territoire.	La valeur ne chute pas par myopie.
Transferts multiples	Positions ou plusieurs cases sont alimentees.	Policy non reduite a la capture immediate.
Bidoua	Case 5+ qui vaut plus qu'une petite capture.	Le modele preserve ou construit l'accumulation.
Yinda-Bidoua	Ajout d'une graine qui change non lineairement le potentiel.	Le modele evite les transports destructeurs.
Transport-Bidoua	Sequence qui cree un grenier exploitable.	MCTS et Policy convergent vers la sequence.
Temporalite	Positions ou le bon coup est preparatoire.	Le modele accepte un gain immediat plus faible.

XVII. Decisions, hypotheses, validations

Sujet	Statut	Commentaire
Moteur exact separe du reseau	Decide	Le reseau n'apprend pas les regles.
Policy-Value	Decide	Necessaire pour MCTS/self-play.
SRN relationnel	Hypothese principale	Doit battre MLP et ablations.
MCTS/PUCT	Hypothese forte	Budget et gain a mesurer.
Bidoua en entree directe	Non retenu v0.1	Preferer emergence et analyse.
Taches auxiliaires ZT/TS/TR/TM	A tester	Peut aider ou contraindre trop fort.
Dataset v001 10k	Laboratoire	Pas corpus final d'apprentissage autonome.
Self-play	Source principale	Avec replay buffer vivant et checkpoints historiques.
Warm-start Minimax	Option experimentale	A comparer au RL depuis zero, sans dependance permanente.
Promotion checkpoints	A definir finement	Matches, calibration, positions critiques.

XVIII. Feuille de route

Lot	Livrable	Critere de sortie
1	Audit et tests moteur	Regles deterministes, conservation des graines, cas limites couverts.
2	Baseline MLP documentee	Pipeline train/eval reproductible.
3	GraphBuilder Songo	Relations statiques et dynamiques inspectables.
4	SRN v0.1	Forward pass, loss RL et warm-start optionnel.
5	MCTS v0.1	pi_MCTS produit des distributions de visites valides.
6	Self-play	Replay buffer rempli et reutilisable.
7	Evaluation generations	Tournois contre anciennes versions et Minimax.
8	Positions critiques	Suites Bidoua/Yinda/transport testees.
9	Ablations	Gain du SRN demontre ou remis en cause.
10	Integration Unity eventuelle	Modele exporte et contrat d'inference stabilise.

Annexe A. Hyperparametres initiaux - statut experimental

Les valeurs ci-dessous sont des POINTS DE DEPART POUR BALAYAGE, pas des decisions scientifiques definitives.

Parametre	Point de depart / plage	Statut
hidden_dim	64	baseline
blocs relationnels	2, 3, 4	a comparer
relations	next + prev	baseline
mirror	off / on	ablation
dynamic_sowing	off / on	hypothese forte
pooling	mean / attention	comparaison
optimizer	AdamW	baseline
learning rate	1e-3 puis schedule	a calibrer
batch size	selon memoire (ex. 128/256)	a calibrer
simulations MCTS	budgets croissants	a mesurer
c_puct	balayage experimental	a calibrer
Dirichlet alpha	a calibrer au facteur de branchement	
epsilon bruit	a calibrer	exploration
temperature	schedule par ply	exploration
Value	scalaire + MSE	baseline
Value alternative	WDL + cross-entropy	experience

Annexe B. Differences avec le code actuel

Le code existant contient deja un reseau MLP multi-tetes, un moteur de regles, Negamax/Alpha-Beta, des pipelines de datasets teacher et des scripts d'entrainement.

Cible SRN/RL :

- moteur exact conserve ;
- MLP conserve comme baseline ;
- SRN ajoute comme architecture parallele ;
- MCTS/PUCT ajoute comme recherche candidate ;
- dataset RL principal = (S, legal_mask, pi_MCTS, z_terminal) ;
- self-play + replay buffer vivant ;
- Minimax = baseline/adversaire/warm-start optionnel, pas source de verite finale.

Le corpus de 10 000 positions reste D_LAB : laboratoire de test et d'evaluation. Il n'est pas relabellise pour devenir le corpus principal d'apprentissage autonome.

Conclusion.

Le SRN reste une hypothese scientifique. Sa valeur doit etre demontree contre le MLP, par ablations des relations, par evaluation des generations RL et par tests strategiques Songo.

Annexe C. Algorithmes recapitulatifs

1. Pipeline complet

```
Algorithme Pipeline_SRN_Complet
1. valider le moteur Songo
2. construire GraphBuilder
3. entrainer baseline MLP
4. choisir protocole initial:
    A = RL depuis zero
    B = warm-start supervise optionnel puis RL
5. initialiser champion = SRN initial selon A ou B
6. repeter pour generation k:
    experiences = GenerateSelfPlayGames(champion)
    ReplayBuffer_Insert(experiences)
    candidate = TrainGeneration(champion, replay_buffer)
    decision = PromoteOrReject(candidate, champion)
    si decision = promote:
        champion = candidate
    archiver metrics, config, checkpoint
7. lancer ablations et validation strategique
```

2. Flux d'une position pendant l'entrainement

```
S brut
-> canonicalisation moteur
-> legal_mask
-> BuildGraph
-> SRN forward
-> priors P + value V
-> MCTS
-> pi_mcts
-> action jouee
-> resultat terminal z
-> exemple (S, legal_mask, pi_mcts, z)
-> replay buffer
-> batch training
```

3. Flux d'une position pendant le jeu

```
S brut
-> moteur: legal_mask + etat canonique
-> BuildGraph
-> SRN: pi_theta, V_theta
-> si budget disponible: MCTS(pi_theta, V_theta)
-> action choisie
-> moteur.play(action)
-> nouvel etat
```

4. Points de controle obligatoires

Etape	Controle	Raison
GraphBuilder	Aretes attendues sur positions simples.	Eviter une relation dynamique fausse.
SRN forward	Formes des tenseurs et masque legal.	Eviter politiques sur coups illegaux.
MCTS	Somme des visites et backup signe.	Eviter inversion de valeur.
Replay buffer	Deduplication et z coherent.	Eviter fuite ou cibles contradictoires.
Training	Policy loss, Value loss, entropy.	Detecter collapse ou surapprentissage.
Evaluation	Tournois et positions critiques.	Ne pas promouvoir sur simple loss.

5. Ordre de priorite de mise en oeuvre

L'ordre recommande vise d'abord la fiabilite des contrats, puis l'apprentissage autonome :

1. Verifier le moteur de regles et les invariants.
2. Verifier la symetrie avant toute canonicalisation definitive.
3. Figer le contrat de donnees RL : (S, legal_mask, pi_MCTS, z_terminal, metadata).
4. Construire GraphBuilder avec NEXT/PREV comme baseline et relations experimentales configurables.
5. Implementer le forward SRN et tester formes, masques et gradients.
6. Tester une boucle d'entrainement minimale sur donnees synthetiques ou de laboratoire uniquement pour valider le code ; ce test ne constitue pas la methode d'apprentissage finale.
7. Implementer MCTS/PUCT et verifier selection, expansion, backup et perspective.
8. Generer un petit corpus de self-play et valider pi_MCTS et z_terminal.
9. Lancer l'apprentissage RL avec replay buffer.
10. Evaluer chaque generation contre checkpoints historiques, MLP et Minimax.
11. Realiser les ablations : relations, pooling, MCTS, Value scalaire/WDL.
12. Integrer le modele valide dans le jeu Unity lorsque le contrat d'inference est stabilise.

Principe final :

Le dataset RL n'est pas un fichier prepare une fois pour toutes. Il est produit par le self-play, alimente le replay buffer, puis evolue avec les generations du modele.